uc3m | Universidad Carlos III de Madrid

Bachelor's Degree in Data Science and Engineering
Academic Year 2025–2026

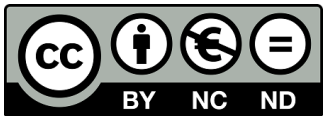
Bachelor Thesis

"Context-Aware Multi-Agent Orchestration: A Standardized Architecture via MCP for Autonomous Agentic Systems"

Arturo Javier Kolster Borges

Boni García Gutiérrez

Leganés, September 2026



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

This thesis presents *thelab-mcp*, a local-first, multi-agent orchestration architecture built on the Model Context Protocol (MCP), a standardized communication protocol between LLM-based agents and local or external data environments. The architecture presented is an autonomous machine learning (ML) factory that converts a local tabular dataset into a versioned, validated, locally served ML model, with a single human approval gate and no mandatory cloud dependency.

The system is designed around the separation of responsibilities. Large language models decide *what* to run, while a deterministic Python pipeline executes *how* it runs. Typed contracts (Pydantic), a SQLite+FTS5 context store, seven MCP servers, an AST-restricted code sandbox, and a single human approval gate connect the agentic layer to the deterministic core. Agents (an analyst, a feature engineer, and a model selector, coordinated through an orchestrator) can only propose and compute inside bounded, sandboxed rounds. Every execution is recorded as evidence in run manifests, every claim an agent makes must resolve to recorded metrics, and a rejected proposal can never be executed. The design principle is *grounded autonomy*. The deterministic factory provides the grounding evidence, and the agents operate on top of it as an additional layer.

The evaluation answers six research questions in two groups. The *system group* is verified by an automated evaluator that runs inside a disposable temporary workspace. (RQ1) A model-training run is reproducible from its dataset, configuration, and seed. (RQ2) An independent MCP client can discover and use a registered model without any knowledge of the training internals. (RQ3) Local context retrieval recovers past evidence without modifying the store. The *agentic group* measures what the agent layer adds on top of that grounding. (RQ4) Context-grounded agents produce more verifiable proposals than ungrounded agents. (RQ5) Sandboxed agent-generated code trains valid, competitive models beyond the fixed deterministic grid. (RQ6) Role-specialized agents coordinated via MCP outperform a single shared-prompt agent.

Different experiments complemented the automated checks. A three-ways-in comparison (CLI, HTTP, MCP) showed identical results across the interfaces. A real-dataset validation demonstrated correct and traceable exploratory data analysis and cleaning policies. Grounded and ungrounded agentic rounds showed that context produces actual verifiable claims instead of guesses. Sandboxed agent code managed to train valid and competitive models beyond the fixed grid. Role-specialized coordination outperformed single shared prompts. Multi-domain stress validation across ten datasets, from genomics and health to crypto, energy, and finance, surpassed the baseline models and generated an absorption loop to improve the deterministic pipeline. A fully-offline execution with a local 3B model demonstrated the architecture's independence from any cloud provider.

The results support the hypothesis. The architecture executes reproducible data-to-model workflows and exposes their artifacts to independent agents without being limited to any specific LLM provider. Its limitations (sandbox filesystem confinement, stdio-only transport, dataset-specific agentic findings) are documented as concrete, traceable findings rather than omitted.

Keywords: Multi-agent systems, Model Context Protocol, reproducibility, machine learning pipelines, interoperability, context engineering, local-first software.

DEDICATION

To my family, especially to my parents, for their unconditional support through these years. To my friends, and to everyone I have met during the bachelor's degree, for making the journey worth it far beyond the classroom. To Universidad Carlos III de Madrid, for being an amazing institution and a great place to learn. And to the open source community, whose tools and generosity have made this work possible.

CONTENTS

LIST OF ABBREVIATIONS xvii
1. INTRODUCTION 1
1.1. Motivation 1
1.2. Problem statement 2
1.3. Objectives 2
1.4. Hypothesis and research questions 3
1.5. Scope 4
1.6. Contributions 4
1.7. Document structure 5
2. BACKGROUND AND STATE OF THE ART 6
2.1. Large language models and agents 6
2.1.1. From transformers to instruction-following assistants 6
2.1.2. Tool use and the agent loop 6
2.1.3. Multi-agent systems 7
2.2. The Model Context Protocol 7
2.3. Context engineering and local memory 8
2.4. MLOps, reproducibility, and provenance 8
2.5. Multi-agent and agent frameworks 9
2.5.1. Benchmarks for ML and data-science agents 9
2.6. Gap analysis 10
3. SYSTEM DESIGN: *THELAB-MCP* 11
3.1. Design principles 11
3.2. Architecture overview 12
3.2.1. The life of a dataset 13
3.3. Typed contracts 14
3.4. The deterministic pipeline 15
3.4.1. Validation as a gate 16
3.4.2. Task-type inference and the model registry 16

3.4.3. The cleaning policy . . . 17
3.4.4. Run artifacts . . . 17
3.5. The context store . . . 17
3.6. The MCP surface . . . 18
3.7. The agent layer . . . 19
3.7.1. Provider abstraction . . . 19
3.7.2. The harness and grounding . . . 20
3.7.3. Worker agent and proposals . . . 20
3.7.4. The approval gate . . . 20
3.7.5. Role agents and the orchestrator . . . 21
3.7.6. The agentic round . . . 21
3.7.7. The chat agent . . . 22
3.7.8. Global agents . . . 23
3.8. Experiment lifecycle . . . 23
3.9. The workspace UI . . . 24
3.10. Safety and security model . . . 27
3.10.1. Alternatives considered . . . 28
4. IMPLEMENTATION . . . 29
4.1. Technology stack and repository layout . . . 29
4.2. Reproducibility engineering . . . 30
4.3. Three ways in: CLI, Python API, HTTP . . . 31
4.4. Kaggle ingestion and context packs . . . 33
4.5. Testing and quality assurance . . . 33
4.6. Development process . . . 34
5. EVALUATION METHODOLOGY . . . 36
5.1. Evaluation objectives . . . 36
5.2. Research questions and pass criteria . . . 36
5.3. The automated evaluator . . . 38
5.4. Experiment 1: three ways in. . . . 39
5.5. Experiment 2: real-dataset validation . . . 39
5.6. Experiment 3: grounded autonomy (the agentic round) . . . 40

5.7. Experiment 4: multi-domain stress validation (the continuous loop) 41
5.8. Metrics 42
6. RESULTS 44
6.1. The recorded evidence base 44
6.2. Experiment 1: three ways in. 45
6.3. Experiment 2: real-dataset validation (S&P 500) 46
6.4. Kaggle pipeline proof 47
6.5. Experiment 3: grounded autonomy (agentic round). 47
6.6. Experiment 4: multi-domain stress validation 50
6.6.1. Per-stage model assignment: the mixed-team experiment 55
6.6.2. GLM-uniform control test: validator as capability enabler 57
6.6.3. Local-first proof: offline execution 57
6.6.4. Three-ways-in prediction sweep 57
6.7. Agent case study: IBM HR attrition 59
6.8. Agent efficiency and reliability metrics 60
7. DISCUSSION 64
7.1. Answering the research questions 64
7.2. Design lessons 65
7.3. Agent reliability in practice 66
7.4. Limitations and threats. 67
7.4.1. System limitations 67
7.4.2. Evaluation threats 68
7.5. Regulatory framework 69
7.5.1. Data protection 69
7.5.2. AI regulation and risk management 69
7.5.3. Licenses and intellectual property 70
7.5.4. Technical standards 70
7.6. Socio-economic environment 70
7.6.1. Budget. 70
7.6.2. Socio-economic impact analysis 71

8. CONCLUSIONS AND FUTURE WORK 73
8.1. Conclusions 73
8.2. Future work 74
8.3. Closing remark 75
BIBLIOGRAPHY 76
A. REPRODUCTION GUIDE 80
A.1. Environment setup 80
A.2. Reproducible training (RQ1 surface) 80
A.3. The three ways in (Experiment 1) 80
A.4. Context store (RQ3 surface) 81
A.5. Automated evaluation 81
A.6. Quality gates 81
A.7. Real-data and Kaggle paths 81
A.8. Troubleshooting 81
B. RUN MANIFEST EXAMPLE 83
C. GENERATED NOTEBOOK EXCERPT 85
D. EXPERIMENT 4 DATASET TABLE 86
D.1. Declaración de uso de Inteligencia Artificial Generativa 88
D.2. Declaración de uso de Inteligencia Artificial Generativa (IAG) en el Trabajo de Fin de Grado (TFG) 94

LIST OF FIGURES

3.1 *thelab-mcp* architecture: three ways in (CLI, HTTP/UI, MCP) over one deterministic core; the agent layer explores, the approval gate decides, the factory executes, and the dashed feedback returns replay-verified champion configurations to the baseline (the ratchet loop). All state lands in local, inspectable stores. 12

3.2 Experiment lifecycle state machine (persisted per experiment). The dashed state is the human approval gate of the agentic round. 24

3.3 Workspace UI: the deterministic Data view, with dataset preview, EDA modules (missing values, distributions, correlations) and the cleaning report. 26

3.4 Workspace UI: the agentic Experiments view, with recorded stage pipeline (Plan, Clean, Train, Evaluate, Agent round), job timeline, best-run card, and the analyst findings brief. 26

6.1 Test-accuracy distribution of completed classification runs. The distribution now spreads across the 0.6–0.9 range on real datasets, with the fixture-driven mass at ≥ 0.99 a visible minority. 45

6.2 Grounding test on the ambiguous schema. Share of verified claims per arm on the raw S&P 500 file. 49

6.3 Per-dataset recorded agentic-round wall-clock (sum of round seconds across all generations; the S&P 500 dataset dominates at 1,995 s). 53

6.4 Ratchet uplift per dataset: deterministic try-all baseline (gray) vs absorbed champion (blue); deltas annotated. 53

6.5 Arm B: best value per driver team and dataset (accuracy for kidney/clinvar/crypto, R^2 for energy). No team dominates: commercial and open teams each won two datasets, llama-uniform the other two. 57

6.6 Prediction latency per surface over the 17 absorbed champions (CLI, HTTP, MCP; milliseconds). Inference is deterministic and token-free on every surface. 58

6.7 The grounded chat agent enforcing the metric-evidence gate live: the agent cited `test_accuracy`=1.0 for a run whose recorded evidence is 0.8045, and the grounding layer refused the answer instead of showing it (red). 60

6.8 Feature-engineering transform cost per driver: completion tokens per round with provider-reported cost (deepseek-v4-flash reasoning burn vs GLM). . 61

LIST OF TABLES

2.1 Multi-agent frameworks compared with *thelab-mcp*'s approach. 9

3.1 Module map of the `thelab` package. 13

3.2 Registered model families and their guards. 16

3.3 The seven MCP servers and their tools. 19

3.4 Safety mechanisms: guarantees and limits. 27

4.1 Core dependencies (versions pinned in `requirements.lock`). 29

5.1 System group (RQ1–RQ3): method and pass bar. 37

5.2 Agentic group (RQ4–RQ6): method and pass bar. Results are recorded separately per provider. 37

5.3 Experiment 3 recorded results (live providers; full tables in Section 6.5). . 41

6.1 Recorded run inventory by status and model family (1,596 runs, Aug 19 – Sep 6, 2026, from `runs/` manifests). 44

6.2 Experiment 1: identical run through three interfaces (iris, seed 42). 45

6.3 Experiment 1, provider dimension: identical outcomes across LLM brains (Titanic, seed 42). 46

6.4 Experiment 2 recorded results (S&P 500 analyst ratings, 164,231 $\times$ 18). . 46

6.5 Kaggle proof: three public datasets, end-to-end. Run identifiers refer to directories under `runs/`. 47

6.6 Experiment 3 / RQ4: grounded vs. ungrounded proposals (live GLM 5.3 Flash; verified-claim rate computed against persisted metrics). 48

6.7 Grounding test on the ambiguous schema (raw S&P 500 file, GLM 5.3 Flash). A claim counts as verified when it resolves to a fact of the file. . . 48

6.8 Experiment 3 / RQ5: generated-code validity and competitiveness (agentic batch entries per dataset). 49

6.9 Experiment 3 / RQ6: multi-agent round vs. single shared-prompt agent (live GLM 5.3 Flash). 50

6.10 Experiment 4, Arm A: recorded deterministic baselines vs. agentic round. The seed column is the winning random seed, recorded so every result is replayable. 51

6.11 Experiment 4, Arm B: stress-domain loop results (mixed teams). 52
6.12 Ratchet loop per generation for ClinVar. 54
6.13 Ratchet loop per generation for Kidney. 54
6.14 Ratchet loop per generation for crypto. 54
6.15 Ratchet loop per generation for energy. 55
6.16 Per-round capability matrix on titanic (provider/model per round, from `stage_models` provenance). 56
6.17 Three-ways-in prediction sweep on absorbed champions: every absorbed model is discoverable via MCP and predictable via CLI, HTTP, and MCP with identical results. Inference is LLM-token-free. 58
6.18 Agent task success over recorded benchmark, case-study, and live agentic sessions. 60
6.19 Recorded token usage per sub-agent interpretation (Titanic benchmark; from provider usage reports). 61
6.20 Model latency probe on OpenRouter with realistic generation. 62
6.21 P6 findings harvest: major findings and by-design rejections (full log in `docs/P6_FINDINGS.md`; every ticket cites its run evidence). 63

7.1 Development budget (recorded costs over the project window). 71

A.1 CLI troubleshooting (from `docs/CLI_GUIDE.md`). 82

D.1 Experiment 4 datasets (Kaggle slugs, verified live; tiered). 86

LIST OF ABBREVIATIONS

AC	Acceptance Criterion
AI	Artificial Intelligence
ANSI	American National Standards Institute
API	Application Programming Interface
AST	Abstract Syntax Tree
CC	Creative Commons (license family)
CLI	Command-Line Interface
CSV	Comma-Separated Values
DTO	Data Transfer Object
EDA	Exploratory Data Analysis
FTS5	Full-Text Search engine, version 5 (SQLite module)
GDPR	General Data Protection Regulation (EU) 2016/679
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JSONL	Line-delimited JSON
LLM	Large Language Model
LOC	Lines Of Code
MCP	Model Context Protocol
ML	Machine Learning
MLOps	Machine Learning Operations
PII	Personally Identifiable Information
PoC	Proof of Concept
PRD	Product Requirements Document
RAG	Retrieval-Augmented Generation
RLIMIT	Resource limit (POSIX `setrlimit`)
RQ	Research Question
S&P	Standard & Poor's (financial data provider)
SSE	Server-Sent Events
STDIO	Standard Input/Output (transport)
SVM	Support Vector Machine
TS	TypeScript
UC3M	Universidad Carlos III de Madrid
UI	User Interface
XSS	Cross-Site Scripting

1. INTRODUCTION

1.1. Motivation

Machine learning (ML) work has traditionally been organized around notebooks, scripts, and interactive experimentation. The arrival of large language models (LLMs) capable of using tools and reasoning over several steps opened a new possibility in this landscape: software in which LLM-based agents read data, propose experiments, run code, and interpret results with some degree of autonomy. The most common commercial platforms for this new wave of *agentic* ML share three traits: they are mostly cloud-first, they keep context, prompts and data on external infrastructures, and reproducibility is a best-effort property rather than a guarantee. Familiar examples are OpenAI's ChatGPT data-analysis mode, Anthropic's Claude Code, and the agent tooling of cloud ML suites such as Google Vertex AI and Databricks.

The Lab defends three design positions: produce local context that both humans and agents can use; execute through a deterministic pipeline instead of letting agents decide all the steps; and use the Model Context Protocol (MCP) as the standard way to share that context, so the base stays open to any provider. Cloud and local models are supported, making the architecture model agnostic.

The concrete result is *thelab-mcp*: a multi-agent orchestration system built with the MCP protocol that ingests and converts data into versioned, validated and locally served ML models, recording every intermediate step as inspectable evidence. We can summarize the product with a metaphor that also names it: the infrastructure is *the lab*, and the models, agents, and applications it produces are *the experiments* run inside it.

One design rule was present through the whole development of the project, from the first prototypes to the current version: *LLMs decide what to run; deterministic code executes it*. The idea behind it was to create an agentic layer on top of a deterministic pipeline. In this case, for machine learning model generation, it provides LLM agents a set of predefined deterministic tools to choose from. These range from data ingestion, exploration and cleaning to model training, evaluation, validation and hosting.

Agents can propose experiments, interpret results, write sandboxed code, and answer questions. Training and metric computation are reserved for the deterministic Python pipeline, and everything an agent starts sits behind one human approval gate. This rule is what makes reproducibility and auditability possible.

1.2. Problem statement

The problem addressed by this thesis can be stated as follows: *design, implement, and evaluate an orchestration architecture that allows LLM-based agents to participate in a data-to-model ML workflow while keeping the workflow reproducible and auditable*.

Three sub-problems can be decomposed from the statement:

- **Reproducibility.** A model-training run must be exactly reproducible from its dataset, configuration, and random seed. The context needed to reproduce the run is then stored in machine-readable form.
- **Interoperability.** An independent agent (one that does not share code with the system) must be able to discover and use the produced models through a standard protocol, without being granted access to the training pipeline internals.
- **Useful context.** The system must record what has happened during sessions (runs, decisions, errors, agent sessions) without leaking any secrets and without corrupting records.

Agentic behavior is non-deterministic by nature. The answer this thesis explores is not to make agents deterministic, but to bound them. Agents work on typed contracts. Agent-initiated execution needs human approval. Generated code runs only inside a restricted sandbox, and its output is validated deterministically before it can reach training. And every claim an agent makes is checked against recorded evidence.

1.3. Objectives

The four registered objectives are the following:

- (1) a *state-of-the-art analysis* of context engineering and standardized communication protocols for AI agents;
- (2) an *architectural framework*: a scalable, MCP-based architecture that splits agent logic from data sources;
- (3) *experimental validation* through a proof of concept in a real-world technical scenario;
- (4) a *performance evaluation* analyzing the impact of the orchestration on context optimization, token efficiency, success rates and costs.

The general objective of this thesis is to fulfill these objectives through an autonomous ML factory, autonomous in its data-to-model execution and human-gated at every agentic boundary, used as the architecture's demonstrator.

More specific objectives can be derived from the ones above:

1. **O1, Deterministic factory (reg. objective 2, 3).** Implementation of a deterministic, local data-to-model pipeline (validation, profiling, contract generation, training, revalidation, registration) that produces a complete, self-describing artifact set per run.

2. **O2, Typed contracts (reg. objective 2).** Defined typed contracts (`TaskSpec`, `RunManifest`, `ArtifactRef`, `DatasetSpec`, `ModelSpec`, `LogEntry`) that every component, human or agent, has to use to exchange information (decoupling agent logic from data sources).

3. **O3, MCP surface (reg. objective 2).** Expose the factory through Model Context Protocol servers so independent agents can discover and interact with datasets, models, artifacts, and context without importing pipeline code.

4. **O4, Context store (reg. objective 2, 4).** Implementation of a local, redacted, searchable evidence store (SQLite + FTS5) with read-only retrieval. Every entry is stored with a fingerprint, so the same event is never duplicated or overwritten. This store is the context-management layer of the architecture.

5. **O5, Agentic layer with human oversight (reg. objective 2, 3).** Implementation of a provider-agnostic agent layer (role-specialized analyst, feature-engineer, and model-selector agents coordinated by an orchestrator, plus a grounded chat agent) that proposes experiments, computes in a sandbox, and records its activity.

6. **O6, Evaluation (reg. objective 1, 3, 4).** Execute the state-of-the-art analysis (Chapter 2); formulate the six research questions; verify them with an automated evaluator; complement them with experiments on real and public datasets at increasing scale; and characterize agent reliability with task success rates and token-efficiency measurements (Section 6.8).

1.4. Hypothesis and research questions

The working hypothesis of this thesis is:

> A local multi-agent orchestration architecture based on typed contracts and MCP capabilities that can execute reproducible data-to-model workflows and expose the resulting artifacts to independent agents without being coupled to a specific LLM provider.

The hypothesis can be operationalized as six research questions in two groups. The **system group** verifies the properties any agentic layer depends on:

RQ1 (Reproducible run). Can a model-training run be reproduced using the same dataset version, configuration, dependency versions, and random seed?

RQ2 (MCP interoperability). Can an independent MCP client discover and use a registered model without knowing the internal training-pipeline implementation?

RQ3 (Context retrieval). Can local structured context retrieval recover useful information about previous runs, decisions, errors, snippets, and artifacts?

The **agentic group** measures the value the agent layer adds on top of that grounding:

RQ4 (Grounding). Do agents grounded in the local context store produce more verifiable proposals than ungrounded agents?

RQ5 (Agentic capability). Can sandboxed agent-generated code train valid, competitive models beyond the fixed deterministic grid?

RQ6 (Orchestration value). Do role-specialized agents coordinated via MCP outperform a single shared-prompt agent?

1.5. Scope

The system works on *tabular* machine learning on a single machine: classification and regression over CSV datasets, with nine scikit-learn model families, ingestion of Kaggle datasets with automatic context packs, a local context store, and a web workspace. On top of that deterministic core sits the agentic layer (Chapter 3): agents explore data inside a code sandbox while training, metric computation and validation remain deterministic.

Deep learning and GPU training, trading and order execution, multi-user deployments, public cloud hosting, and language-model fine-tuning are out of scope for this project. However, one boundary stays open on purpose: the sandbox limits the time, memory, and libraries of the generated code, but it does not isolate that code from the files of the machine. Real file isolation needs operating-system support, and it is documented as future work. The decision and its risks are documented in the limitations (Section 7.4).

1.6. Contributions

The contributions of this thesis are:

1. **An architecture** for auditable agentic ML. LLMs make the decisions and a deterministic pipeline executes them, connected through typed contracts and a single human approval gate. On top of it, a grounded autonomy round lets role-specialized agents explore beyond the deterministic grid. The agents start from the pipeline's artifacts, and their output is validated deterministically.

2. **A working implementation** of approximately 16,900 lines of Python plus a React/TypeScript workspace. It includes the deterministic factory, seven MCP servers, a redacted context store, a provider-agnostic agent layer with per-role prompt contracts, and an AST-restricted code sandbox. It is verified by 546 automated tests.

3. **An automated evaluation protocol** (`evaluate_thesis.py`). Every check runs inside a disposable temporary workspace. RQ1–RQ3 run on every demand, and the agentic questions (RQ4–RQ6) use comparison runs that measure what grounding, generated code, and role specialization each add. Four recorded experiments complement it on fixture, real and public (Kaggle) data, comparing different LLM providers.

4. **An honest record of limitations**. It includes an adversarial audit of the full system, more than 500 checks across the CLI, HTTP service, MCP surface, and sandbox, plus an internal review of the project's agentic performance, which found five defects (an SSE serialization crash, hyperparameter-grid misapplication, a fixtures-cleaning restriction, a phantom approval principal, and a feature-name mismatch), all fixed and regression-tested. The findings, both fixed and open, are documented as evidence.

1.7. Document structure

The rest of this document follows the IMRAD convention.

Chapter 2 reviews the technical background (LLM agents, MCP, context engineering, MLOps) and related work (multi-agent frameworks, agent benchmarks, experiment-tracking platforms), closing with the gap this thesis addresses. **Chapter 3** presents the design of *thelab-mcp*: principles, architecture, contracts, deterministic pipeline, context store, MCP surface, agent layer with its approval gate and agentic round, and safety model. **Chapter 4** describes the implementation: the three usage surfaces, reproducibility engineering, and the development process itself. **Chapter 5** defines the evaluation methodology: the six research questions, pass criteria, the automated evaluator, and the four experiments. **Chapter 6** reports the results, deterministic (RQ1–RQ3) and agentic (RQ4–RQ6). **Chapter 7** discusses the findings, the limitations and threats, the regulatory framework (Section 7.5), and the socio-economic environment (Section 7.6). **Chapter 8** concludes and outlines future work. The annexes also contain a reproduction guide and a real run manifest.

2. BACKGROUND AND STATE OF THE ART

This chapter reviews the concepts on which *thelab-mcp* is built and the prior work it relates to. Sections 2.1–2.2 cover large language models, tool-using agents, and the Model Context Protocol. Section 2.3 discusses context engineering and why a local, verifiable context store is used. Section 2.4 reviews reproducibility and experiment tracking in MLOps. Section 2.5 compares multi-agent frameworks, and Section 2.6 states the gap this thesis addresses.

2.1. Large language models and agents

2.1.1. From transformers to instruction-following assistants

The story of modern large language models begins in 2017, when the Transformer architecture [1] replaced the older recurrent designs with a mechanism based on attention. It turned out to work and scale surprisingly well, setting the base architecture for future work in the artificial intelligence field. The next chapter was size: GPT-3 [2] showed that training even larger Transformers on more text produced few-shot learning, meaning the model could attempt tasks it was never explicitly trained for. But a big model that only completes text is not an assistant, so the following step was behavior: instruction tuning [3] aligned raw models to follow user instructions, producing the assistant-style systems that agentic applications are built on. The last piece was reasoning. Chain-of-thought prompting [4] showed that asking the model to write out intermediate steps highly improved multi-step tasks, and multi-step reasoning is exactly what an agent does when it plans an experiment.

2.1.2. Tool use and the agent loop

Talking was only the first capability, and on its own it was not very useful: a model that writes perfect instructions cannot read your data or run your code. The fix was giving the models access to tools. ReAct [5] showed in 2022 that the strongest pattern interleaved reasoning with action: the model thinks, calls a tool, reads the result, and thinks again. Toolformer [6] went one step earlier in the pipeline and showed that models could even learn *when* a tool call is needed, from self-supervised data. From these results the standard agent design was created. So from it, we can define an agent as a system composed of an LLM, a harness that mediates its execution, and the set of tools it may call [7]. The model receives a conversation history and a list of tool specifications, it either writes text or emits structured tool calls, the harness executes them and appends the results, and the cycle repeats until a final answer or a step budget is reached. Reflexion [8] closed the loop

by showing that failures fed back to the models as context improved the next attempts. For this thesis the takeaway of the whole line of work is simple: what an agent *knows* is decided by what its harness supplies, the set of tools and the specific context.

2.1.3. Multi-agent systems

Once single agents worked, the natural next idea was to give them roles. MetaGPT [9] assigned software-company positions to agents and reported more coherent output, and generative-agent simulations [10] showed that memory and reflection produce believable behavior with fewer errors. With role specialization each agent narrows its prompt, tools, and grounding to one responsibility. The problem with more agents working together is the coordination complexity it adds to the equation as well as a larger surface through which errors can spread out. *thelab-mcp* takes a middle position: each agent has a role, a distinct prompt and an allowed tool list; generated code runs only in the sandbox and is validated deterministically, and training itself always stays in the deterministic factory (Section 2.5 compares this with heavier frameworks).

2.2. The Model Context Protocol

By 2024 agents were multiplying, and so was a practical problem: every integration between an application and a model was custom work. In November 2024 Anthropic published the Model Context Protocol (MCP) [11], [12], an open protocol that standardized the way applications communicate with LLMs and provide context and tools. It follows a client–server model: a host application (an agent, an IDE extension, a CLI) embeds an MCP client that connects to one or more MCP *servers*, each exposing three primitives: *tools* (model-invoked functions with JSON schemas), *resources* (application-driven data), and *prompts* (user-selected templates). Messages are JSON-RPC 2.0, and the reference transports are `stdio` (local servers spawned as subprocesses) and HTTP with server-sent events (remote servers).

MCP matters for this thesis for three reasons. The first is interoperability, which is actually the base of the protocol: “Any client that speaks MCP can use an MCP server”, without sharing code with it and without knowing how it works inside. A model registry exposed over MCP can therefore be consumed by an agent that knows nothing else about the system. The second is typed tools. Every tool declares an input schema, so the requests of the model become validated as structured calls. In *thelab-mcp* the schemas also set the boundaries and reject unknown fields, via the typed-contract philosophy of Chapter 3. The third is local deployment. The `stdio` transport runs each server as a small program on the user’s machine, so the shared context stays local.

2.3. Context engineering and local memory

Once agents started doing more and more real work, people realized that one of the main issues was deciding what information goes into the model's context window. This practice was named context engineering [13], [14], and its central question is memory: what happened before, and how can an agent recover it?

The mainstream answer is retrieval-augmented generation (RAG): embed documents into a vector database, retrieve by semantic similarity, and inject the results into the prompt. RAG is a powerful solution that works really well with LLM-based agents, but for this project it brought costs that did not fit the scope. As it usually adds a cloud or heavyweight local dependency, its retrieval quality is hard to verify, and its results are not easy for a human to audit by inspecting the store.

So the decision was to use a simpler solution: a local, structured, full-text context store. Evidence (run manifests, decisions, errors, agent session summaries) is written to SQLite with an FTS5 full-text index [15], redacted at write time, fingerprinted with SHA-256 so the same event is never duplicated or overwritten, and retrieved read-only. Search is keyword-based with structured filters (run id, tags, event type, time range). Every stored entry is inspectable, every search is reproducible, and the store's bytes are unchanged by reads, a property that RQ3 asserts. Semantic hybrid search is part of future work (Section 8.2).

2.4. MLOps, reproducibility, and provenance

In machine learning, reproducibility is a foundational requirement of experimental work, and is a well-documented field. For example, the JMLR reproducibility program [16] turned the problems that appear in practice into concrete checks for published papers. The industry answer was experiment tracking: platforms such as MLflow [17] and DVC [18] persist parameters, metrics, and artifacts per run.

These three practices were part of the inspiration of the project. The run is the unit of record: every training execution produces a run directory with a manifest (inputs hash, configuration, seed, dependency versions, statuses), metrics, and artifact references, the same mental model as MLflow's runs but implemented with local files and JSON instead of a server. The seed is the mandatory, recorded argument of every run. It is injected into every estimator, and the evaluator verifies that two identical seeded runs reproduce identical metrics. Finally, validation works as a gate: a model is only registered and servable after its run is validated and approved.

Additionally, real-world tabular data adds one more problem: *leakage*, features that encode information not available at prediction time, which inflate metrics and degrade production performance [19]. The cleaning policy of Chapter 3 and the leakage study in Chapter 6 deal with this problem concretely.

2.5. Multi-agent and agent frameworks

The frameworks in Table 2.1 arrived from the orchestration side of the story, and they solve real problems: AutoGen [20] made LLM-to-LLM conversation programmable, LangGraph [21] turned agent flows into stateful graphs, CrewAI [22] packaged role-based crews, PydanticAI [23] added typed tool layers, and Semantic Kernel [24] connected the Microsoft plugin ecosystem.

System	Model	Local-first	Provider	Auditability
AutoGen [20]	Conversable agents; LLM-to-LLM chat	Partial (cloud LLMs typical)	OpenAI-style	Conversation logs
LangGraph [21]	Stateful graphs of agent nodes	Partial	Agnostic via LangChain	Graph state/checkpoints
CrewAI [22]	Role-based crews	Partial	Agnostic	Task outputs
PydanticAI [23]	Typed agent + tool layer	Partial	Agnostic	Typed results
Semantic Kernel [24]	Planners + plugins	Partial	Microsoft-centric	Plugin traces
thelab-mcp	Contracts + deterministic factory; bounded agent round	Context and evidence local	Agnostic (4 adapters)	Manifests + redacted store

Table 2.1. MULTI-AGENT FRAMEWORKS COMPARED WITH *THELAB-MCP*'S APPROACH.

Most frameworks orchestrate *LLM calls* and assume cloud LLM access as the default deployment. However, *thelab-mcp* orchestrates *work*, of which LLM calls are one part, alongside the deterministic pipeline, context store, and MCP servers. It also allows local LLM wiring and even works without any LLM provider. The deterministic pipeline can run without agents: agents are a layer on top of the pipeline, not the whole pipeline.

2.5.1. Benchmarks for ML and data-science agents

Another line of work tests agents as *machine-learning practitioners*. MLE-bench puts them on Kaggle-style ML engineering tasks [25], DS-1000 tests data-science code generation [26], DSBench and TML-Bench measure end-to-end data-science and tabular-ML work [27], [28], and AstaBench covers the scientific workflow [29]. With this project the aim was slightly different. The question was not so much whether the agent can win the task, but whether it can win and the system can prove what happened. Do runs reproduce, can an independent client consume the results, and does the evidence survive retrieval intact? The two approaches complement each other, and a capability benchmark is the

natural next evaluation move (Section 8.2). Additionally, the Kaggle-style task formats directly inspired the Experiment 4 design.

2.6. Gap analysis

The reviewed landscape exhibits a clear gap. Experiment trackers (MLflow, DVC) provide provenance but no agentic participation. Agent frameworks provide orchestration but treat the ML workflow as conversation, with weak provenance and cloud-first defaults. Commercial agentic-ML products combine both but restrict the user to external infrastructure and opaque execution. What is missing, and what this thesis tries to contribute, is a system that is simultaneously:

- **complete** end-to-end (dataset → validated model → served inference), not a component library;
- **evidence-local** (datasets, run evidence, and context stay on the user's machine; every LLM provider is optional and swappable);
- **auditable by construction** (typed contracts, manifests, redaction, approval records, rejection-as-evidence); and
- **interoperable** through a standard protocol (MCP), so independent agents can consume the factory safely.

thelab-mcp is designed to occupy exactly this intersection, and its behavior has been empirically validated during development across recorded runs on real data, including failed rounds and honest refusals.

3. SYSTEM DESIGN: *THELAB-MCP*

This chapter presents the design of *thelab-mcp*. Section 3.1 states the design principles; Section 3.2 gives the architecture overview; Sections 3.3–3.6 describe the typed contracts, the deterministic pipeline, the context store, and the MCP surface; Section 3.7 details the agent layer; Section 3.8 covers the experiment lifecycle; Section 3.9 describes the workspace UI; and Section 3.10 consolidates the safety model.

3.1. Design principles

Five principles were established at the start of the project before any code was written and have been preserved throughout the development:

1. **Deterministic core first.** "The first usable milestone must work without a UI, an LLM, or network access." The factory must be complete and useful before any intelligence is layered on; agents amplify the factory but do not substitute for it.

2. **LLMs decide; deterministic code executes.** Language models propose experiments, interpret results, and answer questions. The training pipeline, cleaning, and inference are preserved to run through the same deterministic Python path. This is the invariant that makes RQ1 possible at all: an LLM that trains models itself would inject non-determinism into the very thing being measured.

3. **Agent autonomy.** Agent-initiated execution requires explicit human approval through a single gate and generated code always passes the sandbox and a deterministic validator. Nothing an agent writes is ever executed directly, and a rejected proposal can never be executed at all.

4. **Rejection as a first-class outcome.** A run that fails validation is a valid, traceable result (complete with manifest, validation report, and reason), not an error to swallow or a crash to retry. The system converts its own refusals into queryable evidence.

5. **Local-first and provider-agnostic.** All execution, data, and evidence stay on the local machine; every LLM provider (Ollama, OpenAI-compatible endpoints, OpenRouter, or a deterministic mock) is an optional adapter behind one typed protocol, never a structural dependency.

3.2. Architecture overview

thelab-mcp offers three ways to interact with it: a command-line interface (CLI) for scripting, an HTTP service with a browser workspace for interactive use, and seven MCP servers for independent agents. All three surfaces resolve to the same deterministic pipeline and produce identical artifacts for identical inputs (verified experimentally in Chapter 6). The reason for three interfaces is the audience: humans exploring data, scripts automating it, and external agents consuming it, all reading the same evidence, so no single surface can drift from the truth. Figure 3.1 shows the architecture.

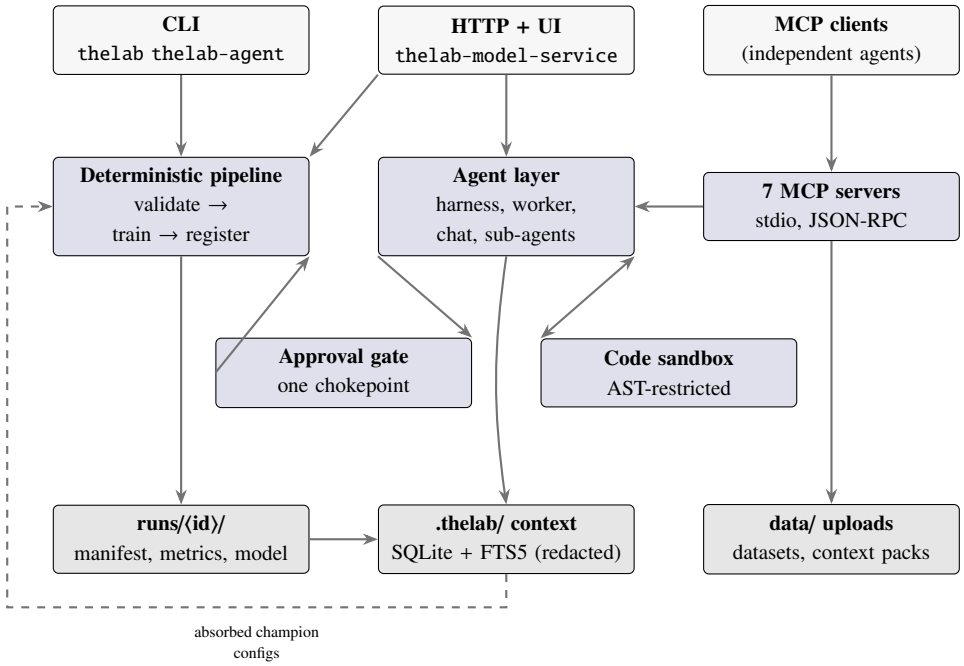


Fig. 3.1. *thelab-mcp* architecture: three ways in (CLI, HTTP/UI, MCP) over one deterministic core; the agent layer explores, the approval gate decides, the factory executes, and the dashed feedback returns replay-verified champion configurations to the baseline (the ratchet loop). All state lands in local, inspectable stores.

The implementation is organized into ten Python packages, summarized in Table 3.1, with approximately 16,900 lines of code in total, excluding the workspace front end.

Package	Responsibility
`contracts`	Six canonical Pydantic typed contracts imported by every other package
`run`	Deterministic factory: validation, profiling, training, artifacts, batch, comparison, notebooks
`context`	SQLite + FTS5 evidence store: repository, redaction, read-only reader, indexer
`agents`	Provider protocol and adapters, harness, worker agent, global agents, chat agent
`ide`	Agentic ML IDE backend: jobs (SSE), orchestrator, datasets, cleaning, experiments, Kaggle
`mcp`	Seven stdio MCP servers, shared path-safety helpers, demo client
`sandbox`	AST-restricted subprocess for LLM-generated code
`model_service`	FastAPI service and dashboard serving the three API surfaces
`eda`	Deterministic exploratory data analysis skill pack (six pure functions)
`workspace`	Path containment and SHA-256 hashing utilities

Table 3.1. MODULE MAP OF THE THELAB PACKAGE.

3.2.1. The life of a dataset

A single worked path explains the architecture better than any component list. The sections that follow describe the pieces. This one follows one dataset through all of them, in the order the system actually executes.

Arrival. A researcher brings a tabular CSV, a bank's customer records, for example, and wants the best predictive model the system can produce and defend. The dataset enters through any of the three surfaces, uploaded in the browser, pointed at from the CLI, or ingested from a public repository. From this moment every object the system touches is a typed Pydantic contract (Section 3.3), never a loose dictionary.

Validation and cleaning. The deterministic pipeline first validates the dataset. The target must exist, the features must be usable, and every class must have enough rows to stratify. A failed check never raises an error to be hidden. It produces a *rejected run*, a complete persisted record with the reason (Section 3.4). The cleaning policy then applies its rules deterministically, from datetime parsing to cardinality-aware encoding and median imputation, and reports every change column by column.

The floor. Before any agent acts, the system trains *every* registered model on the cleaned data with default hyperparameters and persists each run (Section 3.4). This try-all baseline is deliberately agent-free. It is the floor the agentic layer must beat for its work to count, and it is reproducible byte-for-byte from the dataset, the configuration, and a seed.

The agentic round. Now the LLM layer explores. Three role-specialized agents run in sequence (Section 3.7.6), each an LLM with its own system-prompt contract and,

optionally, its own model. The *Analyst* reads the baseline's metrics and prior evidence through read-only MCP tools (Section 3.6). The *FeatureEngineer* writes one pandas transform that executes *only* inside the code sandbox (Section 3.10), and the result is validated deterministically before it may reach training. The *ModelSelector* proposes training configurations beyond the default grid, other families, other seeds, and hyperparameters the deterministic pipeline never tries. Every proposal passes one approval gate. A human says yes, and the record shows who initiated and what was approved.

Training and artifacts. Approved configurations are executed by the same deterministic factory, never by the LLM. Each run lands as eleven artifacts in `runs/⟨id⟩/`, from manifest and metrics to the trained model (Section 3.4). A win counts only after the factory replays the winning configuration with the same seed and reproduces its metrics exactly. Only then is it absorbed as the dataset's champion configuration, and the next generation must beat it.

Memory and serving. Everything is appended to the local context store, from run summaries to round records with the exact prompts and token costs, redacted on arrival and searchable, so the next question can be answered from evidence instead of memory. Champion models are servable through all three surfaces at once. A CLI one-liner, an HTTP endpoint, and an MCP tool return identical predictions, which makes every claim in this thesis independently checkable. Section 5.7 measures what this loop achieves across ten datasets.

3.3. Typed contracts

All information exchanged between components (human, deterministic, or agentic) travels in six canonical contracts built with Pydantic [30] in strict mode (`strict=True, extra="forbid"`), so that unknown fields and automatic type conversions are rejected rather than silently accepted. This choice is the whole motive of the architecture: agents can only communicate through types the whole system already understands, which removes an entire class of integration errors.

The central contract is the run manifest, shown in Code 3.1. It records the input hash, the complete training configuration, the random seed, and the resolved dependency versions: the minimum provenance needed to reproduce and to audit a run.

CODE 3.1. The `RunManifest` contract (`thelab/contracts/run_manifest.py`).

```
class RunManifest(BaseModel):
    model_config = ConfigDict(strict=True, extra="forbid")

    run_id: str
    input_hash: str                        # SHA-256 of the dataset
    training_config: dict[str, Any]
    random_seed: int
    dependency_versions: dict[str, str]
```

```
    started_at: datetime
    finished_at: datetime | None
    final_status: RunStatus              # pending|running|completed
        |failed|rejected
    validation_status: ValidationStatus  # pending|approved|rejected
    artifact_refs: list[ArtifactRef]
    error_summary: str | None
    task_type: Literal["classification", "regression"] | None
```

Artifact references deserve special mention. Every artifact is addressed by a *relative* path and a content hash; a validator rejects absolute paths and any path containing `..` parts (Code 3.2), making traversal attacks unrepresentable in the type system rather than just forbidden by a convention.

CODE 3.2. The `ArtifactRef` contract: relative paths enforced by validation.

```
class ArtifactRef(BaseModel):
    model_config = ConfigDict(strict=True, extra="forbid")

    artifact_id: str
    artifact_type: str
    relative_path: Path    # validator rejects absolute paths and "
        .."
    content_hash: str      # SHA-256
    origin: str
    parent_run_id: str
```

The remaining contracts complete the vocabulary:

- `TaskSpec` (an orchestrator's unit of work, with `objective`, `input_refs`, `constraints`, and `task_state`),
- `DatasetSpec` (source, expected schema, target, quality rules, privacy classification),
- `ModelSpec` (algorithm, hyperparameters, target metric, seed, version), and `LogEntry` (event type, tags, redacted summary, artifact references, privacy level).
- `ExperimentProposal` (the shape an agent's suggestion must take) extends this vocabulary with a goal, a model grid validated against the model registry, seeds, and a rationale grounded in EDA and prior runs.

3.4. The deterministic pipeline

The pipeline converts a local CSV into a registered model through a fixed sequence: *validate* → *profile* → *contract* → *train* → *re-validate* → *register*. Each stage is a pure,

testable transformation and the composition is deterministic given (dataset, model, seed). The order matters: profiling runs before training so that a dataset with silent problems (wrong types, a missing target, constant columns) fails early with a reason, instead of failing mid-training without one.

3.4.1. Validation as a gate

Input validation is needed for the pipeline to work seamlessly without errors. It checks schema conformance, data types, missing values, duplicates, target validity, and split feasibility. It is also useful when the pipeline fails. When a run is rejected (`final_status=rejected`), all the evidence is stored in the same format as successful runs, letting agents have context about failed runs to avoid similar mistakes in the future.

3.4.2. Task-type inference and the model registry

The task type (classification or regression) is inferred from the target column (at most 20 distinct values ⇒ classification) or can be forced explicitly. Nine model families are registered (Table 3.2); each entry carries its estimator class, default parameters, probability support, task type, and an optional *scale guard* `max_train_rows`. When a configuration violates a guard (for example, `svc` on 113,769 rows), the run is rejected *in milliseconds* instead of running for hours and exhausting memory.

Name	Task	Estimator	Guard
`logistic_regression`	classification	`LogisticRegression`	
`random_forest`	classification	`RandomForestClassifier`	
`svc`	classification	`SVC`	≤ 50,000 train rows
`sgd_classifier`	classification	`SGDClassifier`	
`hist_gradient_boosting`	classification	`HistGradientBoosting...`	
`linear_regression`	regression	`LinearRegression`	
`ridge`	regression	`Ridge`	
`random_forest_regressor`	regression	`RandomForestRegressor`	
`hist_gradient_boosting_regressor`	regression	`HistGradientBoosting...`	

Table 3.2. REGISTERED MODEL FAMILIES AND THEIR GUARDS.

The seed is injected into every estimator (`random_state=seed`) and into the train/test split. Preprocessing is a fixed `StandardScaler` pipeline and metrics are computed consistently throughout runs: accuracy and macro-F1 for classification, RMSE, MAE, and R^2 for regression, on both train and test partitions.

3.4.3. The cleaning policy

One of the first problems was that real datasets almost never arrive clean. To make it work, the deterministic cleaning policy turns raw uploads into trainable tables while remaining fully auditable: rows with a missing target are dropped; datetime columns are parsed into numeric calendar features; categorical columns with at most 20 distinct values are one-hot encoded, while high-cardinality categoricals (e.g., 500 ticker symbols) are frequency-encoded; missing numeric values are imputed and constant columns are dropped. Every action is recorded per column in a `cleaning_report`. The importance of the cardinality rule is quantified in Chapter 6: for example, on a real 164,231-row financial dataset, naive one-hot encoding produced a 22 GB intermediate file, while the final cleaning policy produced 16–21 MB with the same raw data.

3.4.4. Run artifacts

Each completed (or rejected) run generates eleven artifacts in `runs/<run_id>/`: `manifest.json`, `task_spec.json`, `events.jsonl`, `inputs.json`, `data_profile.json`, `dataset_contract.json`, `training_config.json`, `metrics.json`, `validation_report.json`, `model.joblib`, and `model_card.md`. This directory is the unit of evidence for the whole system: MCP servers, the registry, the UI, and the context indexer all read from it, and Annex B reproduces a real manifest.

3.5. The context store

The context store is *thelab-mcp*'s memory: a local SQLite database with an FTS5 full-text index, designed around four properties.

1. **Redaction at write time.** To avoid possible sensitive information leakage, twelve ordered regular-expression families strip private key blocks, bearer tokens, GitHub personal access tokens, Google/AWS/Slack API keys, `sk-` keys, passwords, and secret-valued environment variables, replacing matches with `[REDACTED]` before anything is persisted.

2. **Content-addressed idempotence.** Every entry carries a SHA-256 `content_hash` over its canonical JSON content; re-indexing the same event has no effect, and a mismatched hash on the same event id raises an error rather than corrupting the store.

3. **Privacy levels.** Entries are classified `public`/`internal`/`restricted`/`secret`; readers default to the first two, and the restricted tiers require explicit override.

4. **Read-only retrieval.** The reader opens the database with `mode=ro` and `PRAGMA`

`query_only=ON`, so retrieval cannot mutate evidence, a property RQ3 asserts byte-for-byte.

Search is keyword-based (SQLite FTS5, results ranked by relevance) over the redacted summaries, with structured filters for run id, tags, event type, and time range. Entries arrive from three sources: the JSONL event log of runs, agent session summaries sent through the `context_write` MCP server (redacted on arrival), and the Kaggle ingestion, which adds one context pack per dataset.

3.6. The MCP surface

Seven MCP servers expose the factory to independent clients over *stdio*, chosen deliberately for a local-first system. Servers run as plain child processes with no network surface, no ports, and no authentication layer to configure, and the single-job split lets a client connect only to what it needs:

- `data_catalog`: what datasets exist, with their profiles and contracts.
- `model_registry`: the approved models and inference with them. This is the RQ2 surface.
- `workspace`: the runs, their manifests, and their artifacts.
- `context`: search over the evidence store (read-only).
- `context_write`: one tool to append agent session summaries, redacted on arrival.
- `eda`: the six deterministic EDA skills (missing values, correlations, class balance, outliers, leakage suspects, feature types).
- `agent`: orchestration, spawning sub-agents, and training jobs, always behind the approval gate.

Table 3.3 lists the complete tool set (32 tools). All seven servers share the same safety basics: run ids are checked against traversal and dot-entries, every answer uses the same `{ok, data}` / `{ok, error}` format, and every tool declares a bounded schema (for example, queries of at most 200 characters) that rejects malformed calls before they run.

Server	Tools
data_catalog	list_datasets, get_data_profile, get_dataset_contract
model_registry	list_models, get_model_manifest, get_model_card, get_model_metrics, predict
workspace	list_runs, get_run_manifest, list_run_artifacts, get_artifact, read_model_card
context (read-only)	get_context_status, get_context_entry, search_context
context_write	append_session_summary (server-side redaction)
eda	missing_profile, correlation_hints, class_balance, outlier_scan, leakage_suspects, feature_types
agent	orchestrate_experiment, run_full_journey, continue_journey, get_journey_status, spawn_subagent, run_deterministic_skill, run_training_job, get_job_status, log_agent_activity

Table 3.3. THE SEVEN MCP SERVERS AND THEIR TOOLS.

The main separation of powers is carried out by three of the servers. The idea behind it is simple: the ability to read evidence is separated from the ability to change it, and the ability to use a model is separated from the ability to train one. An external agent can therefore look at everything the system recorded, but it can only modify the record through guarded doors.

The read-only `context` server is the first half. It gives any client search over the evidence, with no write path at all: whatever an agent learns from it, it cannot rewrite or delete. The `context_write` server is the guarded door in the other direction. It accepts session summaries only after the same redaction pipeline the store itself uses, so an agent can add to the record but cannot bypass the redaction. Finally, the `model_registry` server is the RQ2 surface: `list_models` and `predict` expose approved models with no reference to the training code that produced them, so inference can be done by an agent that has no idea how the model was built. And because serving is gated by validated state, a model that never passed validation simply does not exist for this server.

3.7. The agent layer

3.7.1. Provider abstraction

Every LLM interaction goes through one small interface. The system asks a provider to complete a conversation, and the provider answers with either text or a tool call. Four providers implement this interface: a scriptable `MockProvider` (used by the entire test suite, making the system fully testable with zero network access), `OllamaProvider` for local models [31], `OpenAICompatProvider` for the de facto `/v1/chat/completions`

standard, and `OpenRouterProvider`. Switching providers is configuration, not code, so the abstraction layer follows the industry-standard protocol itself. This abstraction is what makes the provider experiments of Chapter 6 possible. The same pipeline runs against a reasoning cloud model, a non-reasoning cloud model, or a fully offline local one, without a single code change.

3.7.2. The harness and grounding

The `AgentHarness` runs a bounded tool loop (8 steps by default) over the MCP server connections. It discovers the tools, pins a read-only allowlist, and *grounds* every answer. It also checks each statement against the saved run files before accepting it. Grounding lives in one shared module (`thelab/agents/grounding.py`), so the rules are the same for the harness, the chat agent, and the global agents. A claim counts as grounded only when the run ids it cites really exist and the metrics it quotes match the recorded `metrics.json` values (within a tolerance of 10^{-3}, across ten metric keys). If a check fails, the harness returns a structured refusal (`grounding_failure`) instead of a plausible-looking answer. Without this check, a confident answer citing a metric that does not exist would be indistinguishable from a correct one. With it, overconfidence becomes a refusal the record shows.

3.7.3. Worker agent and proposals

The `WorkerAgent` turns goals into `ExperimentProposal` objects. It feeds the provider a prompt seeded with deterministic EDA results (missing values, class balance, feature types, inferred task type) and the metrics of the last three completed runs on the same dataset and target, so that proposals are grounded in evidence by construction. To make things work and avoid problems at runtime, the LLM provider's output passes through a conservative JSON-repair ladder (markdown-fence stripping, Python triple-quoted-string conversion, trailing-comma and single-quote fixes, unquoted-key quoting) that maintains the required format. If it fails, a deterministic fallback proposal is carried out instead. The worker *never* executes. Proposals are persisted by a `ProposalStore`, and every path that turns a proposal into training must pass the approval gate (Section 3.7.4).

3.7.4. The approval gate

Approval is enforced and carried out in exactly one place, the module `approval.py` in `thelab/agents/`. No other piece of agentic code can approve anything, so there is only one door to check.

- **Agent-initiated flows** (an MCP client asking for orchestration) return `awaiting_approval` by default. An agent cannot silently execute training. Auto-approval

exists only behind an explicit environment flag, and even then it is recorded as `auto:⟨principal⟩`, so the record shows it was not a human decision.

- **User-initiated flows** (a CLI or UI experiment run) carry the initiator's recorded mandate. The deterministic stages are approved automatically as `auto:experiment:⟨id⟩`, so the audit trail shows who started the experiment and that no human reviewed that specific proposal.
- **Rejection is final.** A rejected proposal can never be executed. The gate refuses it outright, closing a loophole in which an approval action could overwrite a rejection.

3.7.5. Role agents and the orchestrator

Three role-specialized agents (`EDAAnalyst`, `FeatureEngineer`, `ModelSelector`) participate in an orchestrated experiment, each with a *distinct system-prompt contract* (`ROLE_SYSTEM_PROMPTS`) that fixes its identity, scope, and output format for the stage. The `ExperimentOrchestrator` sequences the stages (EDA → feature engineering → model selection → approved batch training). Additionally, it applies user feedback to all three stage interpretations and to the proposal goal, over the deterministic pipeline outputs, as well as emitting stage events. These interpretations are persisted alongside the results as searchable context that can be used by the *sub-agents* spawned by the `ExperimentOrchestrator`. If the provider fails, the pipeline continues deterministically, so interpretation never blocks execution (Section 6.8).

3.7.6. The agentic round

Two design decisions define this mechanism. First, the round runs *after* the deterministic batch, never instead of it. The batch's persisted artifacts are the evidence the roles reason over, so the agents are always grounded in something real and the factory's reproducibility is untouched by anything an LLM produces. Second, the roles *propose* but never *execute*: an LLM may decide which model to train, which seeds, and which feature transform to attempt, but the training itself is carried out by the deterministic factory, and every proposal crosses the approval gate before it can run. This separation bounds the "blast radius" of anything non-deterministic. So the worst a hallucinating agent can do is waste compute on a rejected attempt, which is then recorded as useful evidence to later patch the system, making it more robust.

The agentic round is the system's *grounded autonomy* mechanism and its multi-orchestration claim. It is an additional exploration stage that runs *after* the deterministic batch, starts from the artifacts that batch produced, and validates everything deterministically before it can affect training. A context pack (EDA brief, baseline metrics, prior-run evidence, and user feedback, assembled without any LLM) grounds three sequential roles. Each role can

be assigned its own LLM provider (per-stage model assignment). The round's configuration accepts a role-to-provider mapping, so a mixed team (a fast analyst, a reasoning feature-engineer, a cheap selector) runs through the same gated pipeline. Every stage records its assigned model and, where the provider reports them, per-stage token counts and cost, making the role-to-model assignment a measurement rather than an assumed choice.

1. **Analyst**: reads the context store and EDA evidence through read-only MCP tools to explain *why* the baseline performs as it does and what to try next. Output is a typed findings brief (findings, opportunities, risks) with deterministic fallback.
2. **FeatureEngineer**: generates one pandas transform that executes *only* inside the sandbox; the transformed artifact is then checked by a deterministic post-validator (shape and schema, target intactness, constant-feature rejection, the at-decision-time leakage policy) before it may reach training, and metrics are always recomputed by the deterministic factory: sandbox output is never trusted.
3. **ModelSelector**: proposes training configurations beyond the fixed deterministic grid, validated against the model registry and persisted through the proposal store.

Once a proposal is approved, the round runs as a tracked background job. Stage events appear in the UI as they happen, and the job can be cancelled cleanly. The round ends with a comparison record that pairs the best agentic result with the best deterministic one, together with the signed metric deltas, the validity rates, and the rejections. The record is stored with the run artifacts and indexed into the context store. This round is what produces the evidence for RQ4 to RQ6 (Section 5.6).

3.7.7. The chat agent

The grounded chat agent (`POST /agent/chat` and the UI chat drawer) is the user-facing intelligence of the workspace. It runs a bounded loop (12 steps) over eight tools: `search_context`, `list_recent_runs`, `get_run_summary`, `dataset_eda`, `get_dataset_context`, `clean_dataset`, `propose_experiment`, and `run_python` (sandboxed; dataset exposed read-only as `dataset.csv`). Every turn is persisted to the context store as a redacted session summary. The agent's self-description is worth quoting because it states the design in first person: it reads context packs, runs deterministic EDA, explores data hands-on in the sandbox, *proposes* experiments ("I never train models myself"), and searches memory. A human approves the proposal, and the factory is the one in charge of execution.

3.7.8. Global agents

Two read-only question-answering agents complete the agent layer. The `Researcher` answers questions strictly from recorded evidence. It reads allowlisted run artifacts, cites every metric claim to the run id it came from, and refuses to state a number the artifacts do not support. The `DiagnosisAgent` inspects a failed run and produces a follow-up proposal, which crosses the same approval gate as any other proposal before it can run. Both reuse the harness, the grounding rules, and the provider abstraction of the rest of the layer, so they answer identically across the four providers.

They are exposed through the `thelab-agent` CLI, in the researcher and diagnosis modes, and through the Python API. They are deliberately kept out of the workspace UI and the HTTP service. The chat agent of the previous section is the user-facing surface. These two are audit and triage tools, run on demand when a question or a failure needs a grounded answer. Their behavior is verified in the unit suite and demonstrated end-to-end in `examples/global_agents_demo.py`.

```
$ thelab-agent --mode researcher \
    --run-id run-20260905-180636-9a12c48d \
    --question "What accuracy did this run achieve and on what
       dataset? Cite the run." \
    --provider openrouter --model mistralai/mistral-small-24b-
       instruct-2501

{
  "answer": "Run run-20260905-180636-9a12c48d is present in the
     workspace.
            test_accuracy is 0.8028838778953827. Validation passed.
               ",
  "citations": {
    "run-20260905-180636-9a12c48d:test_accuracy": {
      "run_id": "run-20260905-180636-9a12c48d",
      "source": "metrics.json",
      "value": 0.8028838778953827
    }
  },
  "artifacts_consulted": [
    "manifest", "metrics", "validation_report", "data_profile", "
       model_card"
  ]
}
```

3.8. Experiment lifecycle

Orchestrated experiments are state machines stored as JSON by an `ExperimentStore`. Persistence is what makes the machine trustworthy. A crash or restart does not lose the

experiment; any state can be inspected later like any other evidence, and the JSON file uses the same evidence format as everything else. Figure 3.2 shows the states. Two transitions deserve note. First, `completed` may cycle back through `iterating` when the user sends feedback ("focus on tree models"), which starts a new planning phase informed by the previous results. Second, `cancelled` is reachable from `training` through clean cancellation.

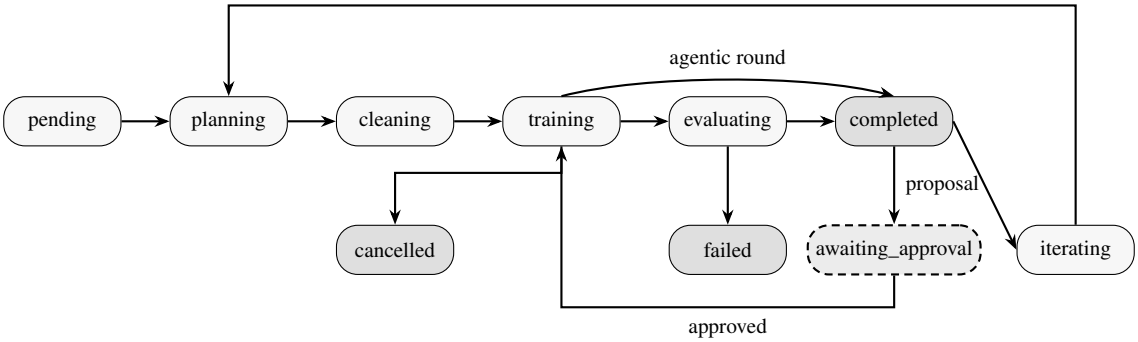


Fig. 3.2. Experiment lifecycle state machine (persisted per experiment). The dashed state is the human approval gate of the agentic round.

Execution runs through a background `JobManager`. It streams per-stage and per-model progress to the UI with server-sent events (SSE). Every event is also appended to the run's `events.jsonl`, so the live view and the later evidence are the same data. Approved agentic rounds (Section 3.7.6) run as further tracked job stages after the deterministic batch and stream their per-agent events through the same channel.

3.9. The workspace UI

The web workspace is built with React 18, TypeScript and Vite, with a hand-rolled CSS token set and Recharts for charts. It is organized as a five-view, experiment-centric layout with a deliberate visual grammar. Blue marks the deterministic side and purple marks the agentic side, so the reader can tell at a glance which half of the system is on screen. Two folders structure the sidebar.

- *Deterministic* holds the Data view and Model Lab. The Data view covers upload of CSV and parquet files, Kaggle import with automatic context-pack build, a sortable preview with horizontal scroll, a grid of EDA cards, and cleaning with an audit report. Model Lab runs the deterministic try-all job with per-model SSE progress and comparison bars.
- *Agentic* holds the Experiments view. It covers planning, running with a live sub-agent stage pipeline, and history with feedback. Proposals live inside the Experiments flow, and approving one launches a first-class tracked experiment with the full stage pipeline and a best-run summary.

- A mini-dock hosts the Global Agent chat drawer and the Admin views. The Admin views include the model registry with metrics, artifacts, predict and evidence tabs, context search with sessions, a sandbox playground, an MCP server inventory panel, and the generated-notebook viewer.

The chat drawer streams per-tool progress, renders markdown, and exposes usage, token and time telemetry for every exchange. It accepts style and role directives, supports inline proposal approval, and keeps the conversation alive across opens and closes. Every exchange is indexed into the context store. Dataset context is shared across views, and the dataset being explored in the Data tab is the same one proposed upon in the Experiments tab.

The first UI was deliberately dependency-free vanilla JavaScript. The rework to React and Vite was a break in that constraint in exchange for maintainability and charting ergonomics. It is recorded here as an example of the project's own willingness to revise its own rules when the evidence favored it.

CODE 3.3. Launching the workspace UI.

```
# one-time: create and activate the environment, install from the
    lockfile
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.lock
pip install -e .

# one-time build of the web front end (React + Vite -> static bundle)
./scripts/build_ui.sh

# launch the service: UI on http://127.0.0.1:8000, JSON API alongside
thelab-model-service --host 127.0.0.1 --port 8000
```

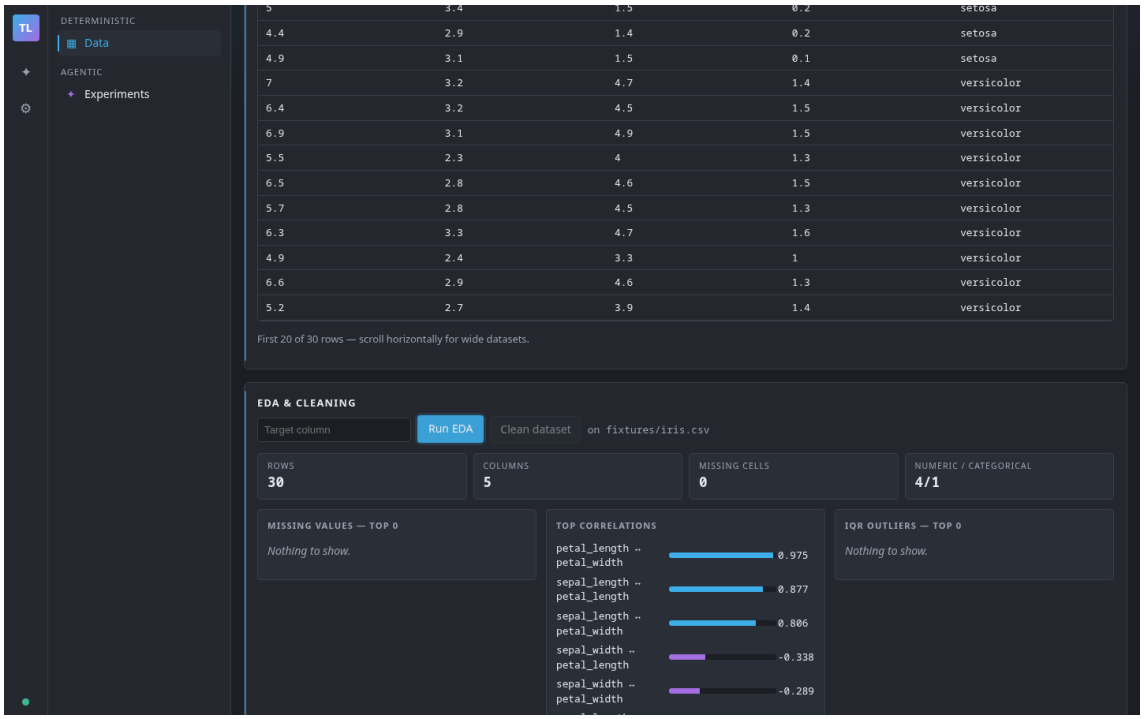


Fig. 3.3. Workspace UI: the deterministic Data view, with dataset preview, EDA modules (missing values, distributions, correlations) and the cleaning report.

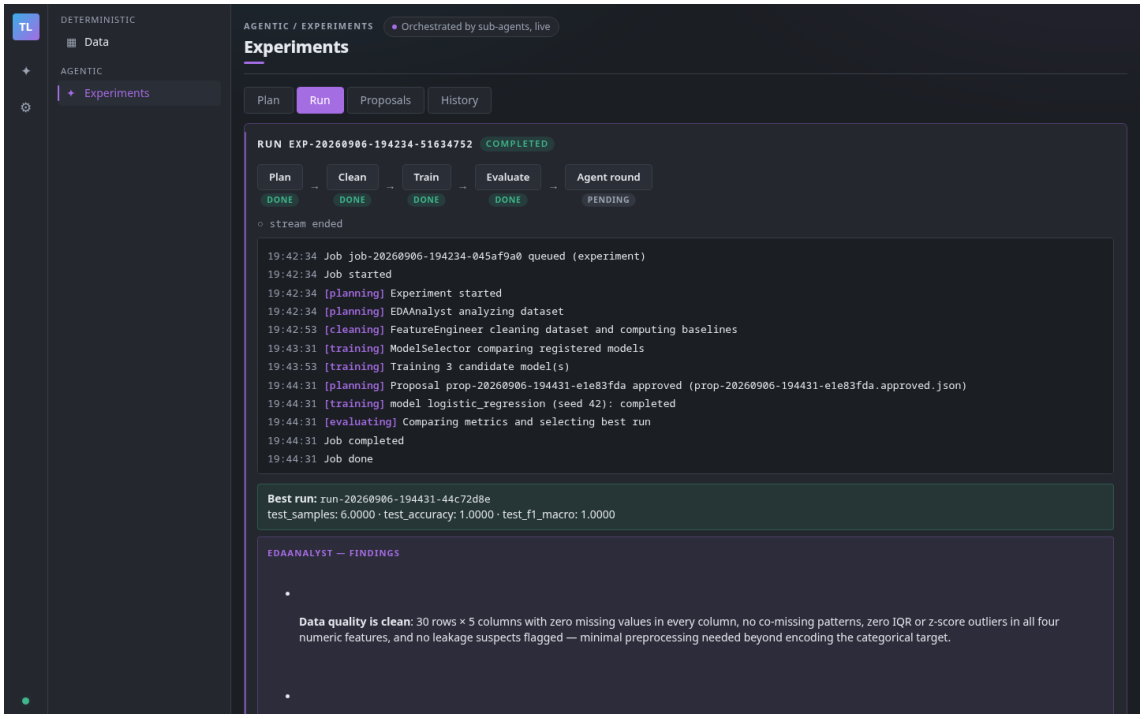


Fig. 3.4. Workspace UI: the agentic Experiments view, with recorded stage pipeline (Plan, Clean, Train, Evaluate, Agent round), job timeline, best-run card, and the analyst findings brief.

3.10. Safety and security model

Three mechanisms define the safety model; Table 3.4 summarizes their guarantees and their honest limits.

Mechanism	Guarantee	Documented limit
Path containment	Workspace-relative paths only; `resolve()+relative_to` blocks traversal and symlinks; `ArtifactRef` validates relativity	Absolute paths may leak in some library error strings (MAJ-01, open)
Code sandbox	Import whitelist (22 modules), 14 blocked builtins, 13 blocked AST node types (lambdas deliberately allowed for pandas idioms), dunder guard, 2 GB RLIMIT, 30 s timeout, output caps	Compute isolation only; no OS-level filesystem confinement (BLK-01, open). The trade-off is deliberate: a local-first, single-user tool avoids requiring container runtimes on the host, and every sandbox artifact is treated as untrusted and re-validated in the parent process
Approval gate	Every path from proposal to training passes one chokepoint; agent-initiated flows default to `awaiting_approval`; rejected proposals are unexecutable	Social, not cryptographic: the principal is recorded (`auto:` mandates included), not authenticated

Table 3.4. SAFETY MECHANISMS: GUARANTEES AND LIMITS.

The sandbox deserves elaboration because it executes LLM-generated code. It spawns a fresh interpreter for every execution and speaks a JSON protocol over stdin and stdout. It forces a non-interactive matplotlib backend and enforces its policy in three layers.

The first layer is a static AST check that rejects forbidden constructs before execution. Class definitions, async constructs, and dynamic `type()` calls are among the rejected shapes. Lambdas are deliberately *allowed*. The RQ5 spike proved that blocking `ast.Lambda` is incompatible with pandas-idiomatic code such as `groupby().transform(lambda s: ...)`. A lambda cannot import, reach blocked builtins, or escape its scope, because `global` and `nonlocal` remain blocked. The second layer is an import hook that re-enforces the whitelist at runtime. The `inspect` module is deliberately excluded, after an audit finding showed it could recover unfiltered builtins. The third layer is compute limits. Memory is capped at 2 GB, raised from 512 MB after measuring the address space pandas 3 needs for its shared libraries. Wall-clock time and output size are capped as well.

The hardening approach follows secure-development guidance on supply-chain and capability scoping [32]. It is aligned with the emerging MCP security literature, the

OWASP MCP Security Cheat Sheet [33], the NSA security guidance for MCP deployments [34], the OWASP LLM risk taxonomy, which treats unrestricted code execution and excessive agency as first-class threat classes [35], and the academic analyses of MCP's attack surface [36], [37]. An adversarial security audit, revisited in Section 7.2, found and fixed two escape chains and documented the remaining filesystem-confinement gap with a proof of concept. That is why this thesis describes the sandbox as providing *compute isolation*, not confinement.

3.10.1. Alternatives considered

Three architectural alternatives were explicitly evaluated and set aside. The record is kept here because it delimits the novelty claim.

- **Orchestration frameworks (Airflow, LangGraph, AutoGen) as the core.** An external orchestrator would compete with the hand-rolled orchestration layer that is itself the thesis's object of study and would dilute the claim. Frameworks remain the comparison baseline of Table 2.1.
- **A workflow/data-transformation engine (dbt) and a warehouse (Snowflake/ BigQuery) as the data layer.** The fit was genuine, since a transformation module wrapped as an MCP tool is exactly the "capability = MCP module" pattern. However, migrating the evidence layer from local SQLite and files to a warehouse conflicts with the local-first motive that defines the system. Recorded as future work for organizational deployments.
- **A vector-database memory layer (RAG) over the context store.** Interesting addition but deferred by decision (Section 4.6). The current keyword FTS5 store preserves verifiability, and semantic retrieval is future work (Section 8.2).

4. IMPLEMENTATION

This chapter describes how the design of Chapter 3 is realized: the technology stack and repository layout (Section 4.1), reproducibility engineering (Section 4.2), the three usage surfaces with working examples (Section 4.3), Kaggle ingestion (Section 4.4), quality assurance (Section 4.5), and the development process itself (Section 4.6).

4.1. Technology stack and repository layout

The stack was chosen for strong typing, determinism, and local operation. Python 3.11+ is the implementation language; Pydantic v2 enforces the contracts; pandas and scikit-learn [38] provide the data and model layers; the official MCP SDK [11] serves the seven stdio servers; FastAPI and Uvicorn serve the HTTP surface [39]; kagglehub ingests public datasets [40]; and SQLite/FTS5 [15] backs the context store. The workspace front end is React 18 + TypeScript built with Vite and served as static files by the same FastAPI process, so the entire system runs as a single local service. Table 4.1 lists the pinned core dependencies.

Dependency	**Role**	**Why this one**
pydantic (≥2)	Contracts	Strictly typed models with validation and the contracts backbone.
pandas (≥2)	Dataframes	Standard tabular layer [41], profiling and cleaning.
scikit-learn (≥1.4)	Models	Nine deterministic estimators and the `random_state` discipline
mcp (≥1.6)	Protocol	Official SDK for stdio servers and clients
fastapi / uvicorn	HTTP	Async service and SSE streaming for job events
kagglehub	Ingestion	First-party Kaggle adapter for public datasets
pyarrow	Parquet	Future-proof columnar I/O support
matplotlib	Figures	Thesis evidence generation (validity charts, comparison plots)

Table 4.1. CORE DEPENDENCIES (VERSIONS PINNED IN `REQUIREMENTS.LOCK`).

The repository follows a conventional Python layout. The package tree maps one-to-one onto the responsibilities of Table 3.1 (packages). The workspace front end lives in `web/`. Every runtime output lands under the user's workspace (`runs/`, `data/`, `proposals/`, `.thelab/`) with persisted references for context searching.

CODE 4.1. Repository layout (top level, abridged).

```
thelab/                   # implementation (10 packages, ~16k LOC)
  contracts/ run/ context/ agents/ ide/ mcp/ sandbox/
  model_service/ eda/ workspace/
  agents/approval.py    # single approval chokepoint
  agents/grounding.py   # shared claim verification
  ide/agentic_round.py  # grounded-autonomy role loop; per-stage
     provider assignment (stage_providers) + token/cost telemetry
web/                      # React + TS workspace (Vite build)
tests/                    # 58 test files, 546 test functions
scripts/                  # evaluate_thesis.py, demo_defense.sh,
   harvest_findings.py,
                          #   probe_model_latency.py,
                             generate_evidence.py, build_ui.sh
examples/                 # fixtures, batch configs, Kaggle experiment
docs/                     # user/CLI/API guides, thesis map, legacy
   records
runs/ data/ proposals/ .thelab/   # runtime workspace (user-local)
thesis/evidence/          # thesis evidence pack (tables, figures,
   snapshots)
```

4.2. Reproducibility engineering

Reproducibility was one of the main goals and is engineered into the pipeline. The four mechanisms that cooperate to make it true are:

1. **Seed injection.** The seed is a mandatory argument of every run, injected into the estimator (`random_state`); `build_estimator` refuses to construct an estimator without it.

2. **Manifest provenance.** The manifest records the input dataset hash, the full training configuration, the seed, and the resolved dependency versions (Python, pydantic, pandas, scikit-learn, numpy, joblib), captured automatically at run time and not maintained by hand.

3. **Lockfile.** `requirements.lock` pins the environment. The documented install path builds the virtual environment from this file, so the recorded dependency versions can be reproduced in practice (Section 7.4).

4. **Generated notebooks.** Every completed run can produce a reproducible research notebook artifact (`report.ipynb`, generated on demand over `GET /runs/{id}/notebook`). The notebook opens with a title and manifest summary, loads the dataset, and includes a reproduce cell that invokes the pipeline with the exact recorded parameters. It also shows the metrics and validation results, indexes the artifacts, and adds the sub-agent findings when present. It is built as standards-compliant nbformat JSON without new dependencies, and is viewable read-only in the UI.

4.3. Three ways in: CLI, Python API, HTTP

The same pipeline is reachable in three different ways. The CLI (Code 4.2) is the primary surface for scripting and the reference for documentation.

CODE 4.2. Reproducible training and prediction from the CLI.

```
# Train one model deterministically
thelab run model --dataset examples/iris.csv --target species \
  --model logistic_regression --seed 42 --output runs

# Output:
# Run completed: run-20260814-222154-a5a01046
#   Model: logistic_regression   Seed: 42
#   Test accuracy: 1.000000      Test macro F1: 1.000000

# One-off prediction from an approved run
thelab predict --run-id <run_id> --features '5.1,3.5,1.4,0.2'

# Index and search the local evidence store
thelab context index --source .thelab/local-logs/agent-events.jsonl
thelab context search "reproducibility" --limit 10

# Agent CLI: worker mode proposes an experiment (EDA-grounded)
thelab-agent --mode worker --dataset examples/iris.csv --target
   species \
  --model-grid logistic_regression,random_forest --seeds 42,43 \
  --provider openrouter
```

The Python API offers the same capabilities to notebooks and scripts (Code 4.3): a low-level `run_model` and `try_all_models`, and a high-level `thelab.quick`.

CODE 4.3. Python API: deterministic run, try-all comparison, and a proposal from a notebook.

```
from thelab.quick import experiment, compare

exp = experiment("examples/iris.csv", target="species", model="svc")
print(exp.metrics)                        # recorded metrics
print(exp.predict([5.1, 3.5, 1.4, 0.2])) # inference from the run

for e in compare("examples/iris.csv", target="species"):
    print(e.model, e.metrics.get("test_accuracy"))

# Agent proposals are importable too -- deterministic fallback, no
   LLM needed
import asyncio
from thelab.agents.mock import MockProvider
from thelab.agents.worker import WorkerAgent
from thelab.ide.proposals_api import approve_and_run_proposal

```

```
worker = WorkerAgent(provider=MockProvider([]), servers=[],
                     proposals_dir="proposals")
p = asyncio.run(worker.propose(goal="Predict species",
                               dataset="examples/iris.csv",
                               target="species",
                               model_grid=["logistic_regression"],
                               seeds=[42]))
outcome = approve_and_run_proposal(p.proposal_id, principal="
    notebook")
```

The HTTP surface serves the UI and a JSON API. Every answer uses the same `{ok, data}` format, so all clients read responses the same way (Code 4.4). Long operations run as background jobs, not inside the request. Training jobs stream their progress live, and experiment jobs move from stage to stage, so a cancel request is cooperative and takes effect at the next safe boundary instead of interrupting a model mid-training. One limitation is documented here: a long synchronous model fit can briefly hold other service traffic (Section 7.4).

CODE 4.4. HTTP surface: training and inference via the local service.

```
thelab-model-service --host 127.0.0.1 --port 8000

curl -s -X POST localhost:8000/train -H 'Content-Type: application/
    json' \
  -d '{"dataset_id": "fixtures/iris.csv", "target": "species",
       "model": "logistic_regression", "seed": 42}'

curl -X POST http://127.0.0.1:8000/predict -H "Content-Type:
    application/json" \
  -d '{"run_id": "run-20260829-...", "features":
       [{"sepal_length": 5.1, "sepal_width": 3.5,
         "petal_length": 1.4, "petal_width": 0.2}]}'
```

Alongside these three, an external agent can use the whole factory through the MCP servers of Table 3.3. It can search the stored context and use the provided tools via the MCP contracts. To prove it works, the repository includes a small demo client that connects to the servers, discovers their tools, and calls them, knowing nothing about the system beyond the protocol itself:

CODE 4.5. MCP smoke tests over stdio.

```
thelab-mcp-demo model_registry --run-id <run_id>  # list_models, card
    , predict
thelab-mcp-demo data_catalog  --run-id <run_id>   # datasets,
    profiles, contracts
thelab-mcp-demo context                           # search the
    evidence store
```

The positioning rule of thumb documented in the user guide is: *use the UI to explore, the CLI to automate, MCP when another agent needs the factory*.

4.4. Kaggle ingestion and context packs

Public data is a first-class input. From a Kaggle slug, the ingestion path (Code 4.6) downloads the dataset into `data/uploads/` through kagglehub (the only networked step and always an explicit user action). It then fetches the dataset's public page context (description, tags, and file metadata, parsed from the page's embedded state), merges it with the local profile (shape, dtypes, head), and saves the merged *context pack* next to the CSV (`<name>.kaggle.json`). The pack is indexed into the context store as a searchable event and exposed to the chat agent through the `get_dataset_context` tool, so proposals are grounded in the dataset's own documentation: for example, ordinal encodings documented on the Kaggle page become part of the agent's evidence, providing useful additional context alongside the raw dataset, so agents can do a better job.

CODE 4.6. Kaggle ingestion via CLI.

```
thelab ingest kaggle pavansubhasht/ibm-hr-analytics-attrition-dataset
# -> downloads CSV to data/uploads/, builds + indexes the context
    pack
```

4.5. Testing and quality assurance

Quality is enforced by four automated layers that run before any change is considered complete to avoid breaking the application code:

CODE 4.7. The four quality gates.

```
.venv/bin/ruff check thelab tests scripts        # lint (E,F,I,UP,W,B
    )
.venv/bin/mypy thelab                            # static types
.venv/bin/python -m pytest tests/ -q             # 546 tests
PATH=.venv/bin:$PATH python scripts/evaluate_thesis.py   # RQ1-RQ6 (
    mock provider)
```

The test suite covers every package at unit level and exercises the full stack end-to-end against the mock provider, so the whole system is testable with zero network access and zero token cost. It includes a real-data hardening suite that replays the S&P-scale scenario, the largest recorded dataset, to make sure the cleaning policy and scale guards keep working at that size. It also includes a suite that tests the thesis evaluator itself as well as different tests that verify the approval boundary end to end (proposal, approve, batch, runs), the sandbox policy against adversarial inputs, the context store's immutability un-

der reads, and the MCP contract of all seven servers. Every named suite can be found in `tests/`: `test_real_data_hardening.py` (S&P-scale hardening), `test_thesis_eval.py` (evaluator), `test_agents_approval.py` (approval boundary), `test_sandbox.py` (adversarial sandbox inputs), and `test_context_reader.py` (byte-identical reads), with the MCP contract of each server covered in its own file, such as `test_mcp.py`.

4.6. Development process

The project development was carried out in seven stages, P0 to P6 (see the README.md and ROADMAP.md). Each stage was subdivided into implementation slices, tested before acceptance, so the application code did not break and the development process did not slow down. Additionally, each stage, when finished, was audited with the help of a custom coding audit agent, which stored the findings in a markdown file to harden the application code, adding patches and fixing bugs.

The seven stages, in order, were the following.

P0, the local-first data-to-model factory. The deterministic core was the first thing built. It included the typed contracts, the training pipeline, four read-only MCP servers, the context store, and the automated RQ1–RQ3 evaluator. The motivation was that the first usable milestone had to work without a UI, an LLM, or network access.

P1, the agentic layer. Intelligence was added on top of that core. This stage brought the provider protocol with its first adapters, the agent harness, the worker agent with experiment proposals, the EDA skill pack, and a cross-domain benchmark, all following one idea: agents propose and the pipeline executes.

P2, the agentic ML IDE. In this stage the factory became usable on real data. It included dataset upload with EDA and cleaning, background jobs with live progress, the code sandbox, and agent orchestration, as well as a hardening phase that added the cleaning policy, scale guards, and cooperative cancellation.

P3, multi-agentic features. The multi-agent claim became real and tangible in this stage. The grounded chat agent could run sandboxed code, ingest Kaggle data with context packs, and generate notebooks. The first multi-Kaggle proof experiment on three public datasets was also carried out in this stage, landing the first results that surpassed the deterministic model baseline.

P4, the workspace rework. The five-view React user interface (UI) was done in this stage; the idea behind it was to have a user-friendly environment that a human would actually want to use apart from the CLI and scripts. In this stage an experiment was also carried out that compared local vs cloud LLM models over the created architecture.

P5, grounded autonomy. In this stage the goal was to expand the multi-agentic features and to generate better and more efficient agent responses overall. Grounded context, per-role agent prompts, the approval gate, and the RQ4–RQ6 instrumentation were built and measured in this stage.

P6, the continuous working loop. Finally, the loop was closed. A multi-domain ratchet loop was done over ten datasets. It compared the metrics achieved by the deterministic pipeline vs the agentic expanded pipeline. If the agent surpassed the baseline, a hole harvester script absorbed the run into fix tickets. So then the deterministic pipeline could be expanded to produce that result with minimal code changes, turning the factory into its own first user and connecting data, models, and software development. Two engineering lessons came out of this stage at real cost: an executor refactor to stop experiment jobs from holding the service event loop was attempted twice and deferred (the direct, blocking pattern was kept for stability (Section 7.4)), and provider latency was measured to be model-dependent in a way that shaped the loop's design, with one reasoning-model call stalling beyond fifteen minutes while non-reasoning models answered in seconds.

5. EVALUATION METHODOLOGY

This chapter defines how the system is evaluated. Section 5.1 frames the evaluation; Section 5.2 binds the six research questions to methods and pass criteria; Section 5.3 describes the automated evaluator; Sections 5.4–5.6 define Experiments 1–3 (deterministic baseline, real-data validation, and the grounded-autonomy round); Section 5.7 defines Experiment 4, the multi-domain stress validation; and Section 5.8 fixes the metrics. All the experiment results are presented in Chapter 6 alongside the findings.

5.1. Evaluation objectives

The goal was to automatically evaluate the hypothesis stated in Section 1.4 with sufficient evidence, *reproducible by construction*. Each check runs automatically, in a disposable temporary workspace, and produces machine-readable verdicts. It also demonstrates the system at increasing scales (fixture, real, and public data) as well as its honesty about what is not yet evaluated (part of future implementation). We can therefore distinguish three evaluation layers:

- **Automated baseline (RQ1–RQ3).** An isolated evaluator runs on every demand, on a small fixture dataset, producing the pass/fail final verdicts.
- **Recorded experiments (Experiments 1, 2, 4).** Three experiments that test the systems claims at realistic scale (three-ways-in, real-dataset validation, and the multi-domain stress validation) with results recorded as regular run artifacts, in the same format the system itself produces.
- **Grounded-autonomy experiment (Experiment 3, RQ4–RQ6).** A component-removal testing protocol that measures what the agent layer adds over the deterministic baseline: proposal grounding, sandboxed generated code, and role-specialized orchestration, saving each model's results separately for later analysis.

The ordering is deliberate: RQ4–RQ6 are asked only after RQ1–RQ3 have established the grounding, measuring what agents add over the deterministic baseline.

5.2. Research questions and pass criteria

The six research questions split into two groups. The system group (RQ1 to RQ3) verifies the properties every agentic layer depends on. The agentic group (RQ4 to RQ6) measures what the agent layer adds on top of that grounding. For each question, the method and the exact pass bar to follow.

Question	How it is tested	What counts as a pass
RQ1, Reproducible run. Can a training run be reproduced from the same dataset, configuration, and seed?	Two `run_model` executions with identical fixture, model, and seed.	Both complete approved. Accuracy and macro-F1 match within 10^{-12}. The manifest records the seed and the dependency versions.
RQ2, Interoperability. Can an independent MCP client discover and use a registered model without knowing the training code?	A separate-process MCP client connects to the model-registry server and runs `list_tools`, `list_models`, and `predict`.	The expected tools are present. The RQ1 run appears in `list_models`. `predict` returns typed, non-empty predictions.
RQ3, Context retrieval. Can local search recover useful past evidence?	One JSONL event is indexed into the SQLite+FTS5 store, then searched through the read-only reader.	At least one relevant hit with stable fields. The database bytes are identical before and after the search.

Table 5.1. SYSTEM GROUP (RQ1–RQ3): METHOD AND PASS BAR.

Question	How it is tested	What counts as a pass
RQ4, Grounding. Do agents grounded in the evidence store make more verifiable proposals than agents without it?	The same proposal task runs twice per dataset, once with the full context pack, once with the evidence stripped away.	Hallucinated claims are reported and proposal validity is at least 90%.
RQ5, Agentic capability. Can agent-written code train valid, competitive models beyond the fixed model grid?	Agents write feature-engineering and training code inside the sandbox. The deterministic validator recomputes every metric outside it.	At least 80% of the generated scripts are valid and there are zero silent failures.
RQ6, Orchestration value. Do role-specialized agents outperform one shared-prompt agent?	The multi-agent round is compared against a single shared-prompt agent, with the deterministic pipeline as the reference.	The multi-agent round completes at least as often as the single-agent one. Every handoff is a typed artifact and the number of human interventions is reported.

Table 5.2. AGENTIC GROUP (RQ4–RQ6): METHOD AND PASS BAR. RESULTS ARE RECORDED SEPARATELY PER PROVIDER.

Note: RQ5 and RQ6 measure *validity, safety, and competitiveness* not so much victory. Improvement over the deterministic baseline is a bonus, not the bar. However after the testing phase most of the agentic runs generated better results than the non-agentic

ones, confirming the hypothesis.

5.3. The automated evaluator

All six research questions are answered by one committed script, `scripts/evaluate_thesis.py`, so any claim in this thesis can be re-checked with a single command. The script runs inside a fresh temporary workspace, prints a verdict table and a JSON summary, and exits with an error code on any failure. No developer state is touched, and the checks are chained, so later questions reuse the runs produced by earlier ones.

The two groups are evaluated in different ways, and the difference is deliberate. The system questions (RQ1 to RQ3) are fully deterministic, so the script automates them completely. RQ1 trains the same model twice through the Python API and compares the metrics at exact-float tolerance (10^{-12}), confirming that the manifest recorded the seed and dependency versions. RQ2 starts a separate MCP client, its own process over stdio, and runs the discovery and prediction round trip against the RQ1 runs. RQ3 indexes one JSONL event and verifies that a search finds it without changing a single byte of the database.

The agentic questions (RQ4 to RQ6) cannot be reduced to one deterministic script, because the thing being measured is the behavior of an LLM. They run as recorded experiments instead. In suite mode, the mock provider drives the rounds, so the instruments (the grounding comparison, the sandbox-code validity check, and the role-assignment comparison) are verified on every demand with zero tokens. In `-live` mode, the same checks run against a real provider, with `-datasets` and `-rounds` selectors for multi-round runs, and the results are recorded as evidence for Chapter 6.

CODE 5.1. Running the automated evaluation.

```
PATH=.venv/bin:$PATH python scripts/evaluate_thesis.py
# ============================================================
# thelab-mcp -- Thesis Evaluation Report
# ============================================================
# Overall: PASS
#   RQ1: PASS   metrics: {'test_accuracy': 1.0, 'test_f1_macro': 1.0}
#   RQ2: PASS   predictions: ['setosa']
#   RQ3: PASS   hits: 1   event_id: evt-repro
```

CODE 5.2. Running the live agentic evaluation, churn dataset, three rounds, abridged.

```
PATH=.venv/bin:$PATH python scripts/evaluate_thesis.py --live
   openrouter --model z-ai/glm-5.3-flash --datasets churn --rounds 3
# ============================================================
# thelab-mcp -- Thesis Evaluation Report, live mode
# ============================================================
# Deterministic baseline, RQ1-verified:
```

```
#   Test accuracy: 0.808   Model: logistic_regression   Seed: 42
#
# Agentic checks RQ4-RQ6 running
#   round 0:  mode agentic   15/15 entries   best 0.8630
#   round 1:  mode agentic    9/9  entries   best 0.8645
#   round 2:  mode agentic    6/6  entries   best 0.8625
#
# RQ4: PASS   grounded claims verified: 1.0
# RQ5: PASS   validity_rate: 1.0
# RQ6: PASS   multi: agentic / single: degraded_deterministic
# Overall: PASS
```

5.4. Experiment 1: three ways in

Claim under test. All three surfaces (CLI, HTTP, MCP) must execute the same deterministic pipeline and produce identical results for identical inputs.

Protocol. On the iris fixture dataset with `logistic_regression` and seed 42: (a) run the CLI command of Code 4.2; (b) submit the same job through `POST /train` on the local service; (c) call `run_training_job` through the agent MCP server and poll `get_job_status`. Record test accuracy, wall-clock time, and the artifact set of each run. **Provider dimension:** repeat the orchestrated experiment on the Titanic dataset (891 rows, target `Survived`) through three execution paths: deterministic-only (no LLM), Ollama (llama3.2:3b), and OpenRouter (GLM 5.3 Flash), and compare metrics bit-for-bit.

Expected outcome. Identical metrics across the three surfaces (1.0000 accuracy on iris, 0.8045 accuracy on Titanic). In this experiment the LLMs models do not create or modify code, they provide a rationale (possible modifications) over the deterministic pipeline, so in this case changing the LLM changes the explanation, not the training outcome. We can also conclude from the results that HTTP and MCP are significantly faster than the synchronous CLI (measured ~2 s vs. ~0.2 s, dominated by interpreter startup, not training).

5.5. Experiment 2: real-dataset validation

Claim under test. The factory behaves gracefully and traceably on real, messy data at larger scales.

Dataset. S&P 500 analyst ratings, 164,231 rows × 18 columns, downloaded from a public source. The dataset was chosen on purpose as it brings real data problems the factory had not ever faced before: mixed types, high-cardinality categoricals (501 tickers and 306 firms), and datetime fields.

Protocol. Five checks, each recorded as runs and artifacts.

1. Run the cleaning policy end-to-end. Measure how rows and columns change, what the `cleaning_report` records, and the size of the produced table against a naive one-hot baseline.

2. Run a feature-horizon leakage study. Train once on naive features and once on at-decision-time features, and compare metrics.

3. Trigger the `svc` scale guard on 113,769 rows. Measure the rejection time and the recorded reason.

4. Run one reproducible real-scale training (logistic regression). Measure time and manifest provenance.

5. Run a full orchestrated experiment (3 models $\times$ 3 seeds) through the HTTP surface. Measure end-to-end time, per-model progress events, and best test accuracy.

Expected outcome. The cleaning policy turns a messy dataset into a cleaned trainable one, with every decision being recorded in the cleaning report. The scale guard rejects impractical models instead of running for hours or exhausting memory. Training at real scale stays fast and fully traceable, and the full orchestration run completes with every model being tracked.

5.6. Experiment 3: grounded autonomy (the agentic round)

Claim under test. Agents grounded in the deterministic baseline's artifacts add measurable, safe and competitive value. Their proposals are verifiable when grounded (RQ4), their generated code is valid and competitive beyond the fixed grid (RQ5), and role-specialized coordination outperforms a single shared prompt (RQ6).

Protocol. Run over at least three datasets (including Titanic and the S&P 500 validation set; iris excluded from RQ5 as it is a fixture dataset, being useful for checking the code wiring but not the logic). Execution is carried out twice and results are recorded separately for local (Ollama) and cloud (OpenRouter) model providers.

1. **Deterministic reference arm.** The standard deterministic experiment pipeline produces the baseline runs, artifacts, and indexed evidence.

2. **Grounded arm (RQ4).** vs **Ungrounded arm (RQ4).** The grounded round runs with the full context pack (EDA brief + baseline metrics + prior-run evidence) and the ungrounded round with the context pack stripped to the minimum schema (no EDA findings, no baseline metrics, no prior runs). The idea is to compare the verified-claim rate (claims resolving to real run ids and recorded metrics), the hallucinated-claim count, and the proposal validity (contract acceptance without repair).

3. **Multi-agent arm (RQ6).** vs **Single-agent control (RQ6).** The comparison is between the three specialized role agentic round of Section 3.7.6 and one individual agent with performing all three stages. Comparing the completion rate, the handoff artifacts, and the final model accuracies.

4. **Generated-code track (RQ5).** The FeatureEngineer transforms and any training scripts execute only in the sandbox; the deterministic post-validator checks shape/schema/target/leakage, and metrics are recomputed outside the sandbox. Record script validity rate, silent-failure count (must be zero), and best-agentic vs. best-deterministic deltas.

Results. Section 6.5 reserves the result tables for the final recorded runs.

Check	Arm	Result	Evidence
RQ4 grounded claims	grounded (EDA context)	4–5/5 verified	raw S&P ambiguous-schema comparison
RQ4 ungrounded claims	zero context / domain prior	0/5 verifiable	honest refusal; hallucinated columns
RQ5 batch validity (iris)	GLM uniform, post-fix	15/15	live OpenRouter recording
RQ5 batch validity (churn)	mixed team ×3	30/30	recorded variance runs
RQ5 batch validity (california)	pre-F4	0/9	honest failure; fix recorded
RQ6 multi vs single	role contracts vs shared prompt	multi agentic; single degraded	GLM shared-prompt control

Table 5.3. EXPERIMENT 3 RECORDED RESULTS (LIVE PROVIDERS; FULL TABLES IN SECTION **??**).

5.7. Experiment 4: multi-domain stress validation (the continuous loop)

Claim under test. The goal was to test the complete stack: Kaggle ingestion, context pack, EDA, clean, agent proposal, approval, train, agentic round, and notebook generation work and generalizes across diverse public datasets without code changes. Ten datasets were selected: six with recorded deterministic baselines (Arm A) and four never seen by the factory (Arm B). All the evidence produced by the experiment lives in `thesis/evidence/raw/` and every number below traces to them.

Protocol. The experiment had two arms:

Arm A: agentic re-run over recorded baselines. Six datasets were used during development to test the architecture (churn, California housing, IBM HR, e-commerce, Titanic, S&P 500). These datasets had only been trained deterministically, so the idea

was to run the architecture this time powered by the agentic layer and compare the results with the already recorded deterministic baselines, comparing best-agentic vs. best-deterministic per dataset, with validity rates and rejections, in order to see if agents produce an improvement in model accuracy.

- (1) *genomics*: ClinVar for gene-variant classification (real-world variant classification, high-cardinality categorical structure).
- (2) *health*: chronic-kidney-disease clinical records (54 clinical features).
- (3) *crypto*: daily cryptocurrency prices (direction prediction across five assets).
- (4) *electrical usage*: 10+ years of hourly PJM consumption (datetime-heavy regression that exercises the cleaning policy's calendar features).

Storage policy. The datasets are kept in the workspace as proof as well as the generated files: the analysis CSV (each under ~20 MB), the context pack, and the cleaning report. Heavy raw sources stay in the transient kagglehub cache and are eliminated after ingest.

The ratchet protocol. The whole idea of the experiment was to generate a working loop (ratchet loop). Each generation starts from a fair deterministic baseline, in which all registered models are trained and persisted by the factory itself. Role-specialized agentic rounds then explore beyond it, trying to beat the model baseline. If an agentic round produces a better model the factory absorbs the model as the winning configuration for the dataset. But it will only be absorbed if it can be reproduced by the factory deterministic pipeline (no code modifications). Champions are recorded in a per-dataset generation ledger (baseline, per-round validity, mode, entries, the prompts, token costs, and the absorption decision with its replay proof), so the next generation of runs will face stronger baselines, and agents will have to produce better results to beat it.

Results. Across the ten datasets the loop recorded **37 generations** and **53 training rounds**, producing **19 absorbed champions**. The remaining 18 generations absorbed nothing, and the ledger records the reason for each one. Five never reached an agentic round (all entries degraded or rejected), two hit a ceiling, where their best possible score already equaled the baseline, and eleven finished below the baseline they faced. The per-dataset results follow: Arm A in Table 6.10, Arm B in Table 6.11.

5.8. Metrics

The supervised metrics are accuracy and macro-F1 for classification, and RMSE, MAE, and R^2 for regression. Macro-F1 is reported because several benchmark datasets are imbalanced, and plain accuracy misleads on them (the IBM HR case in Section 6.7 has an 83.9% majority-class baseline). Reproducibility has its own measure: two runs count as

identical when their metrics match within the evaluator tolerance, 10^{-12} for the automated check. The grounding tolerance, 10^{-3}, applies to agent claims, not to runs. This scope decision is recorded in Section 7.4. Wall-clock times are reported on the development machine (Intel laptop, 16 GB RAM, no GPU), so every result is reproducible on commodity hardware.

The agent-level metrics complete the set. They are required by the registered performance-evaluation objective and by the agentic research questions.

- **Task success rate.** A task counts as successful when the full cycle works. When the agent proposes an experiment, the proposal passes validation, the human approves it, and the deterministic pipeline trains the model to completion.
- **Token efficiency.** Tokens are counted per completed agent task, using the usage data each provider reports. The mock provider and the deterministic fallback use none, so the whole test suite runs at zero cost. Token spending only happens at proposal time, never during training, and it is reported separately for each provider.
- **Verified-claim rate (RQ4).** When an agent answers, it cites run ids and metrics. The verified-claim rate is the share of those citations that check out against the recorded evidence. Anything pointing to evidence that does not exist is reported as a hallucinated claim.
- **Script validity (RQ5).** Agents write code, so the thesis measures how often that code actually works. Script validity is the share of generated scripts that run in the sandbox and pass the deterministic post-validator. A silent failure, code that runs but produces a broken artifact, is counted on its own.

All agent metrics share the same honesty rule: they are computed over recorded, inspectable sessions, never reconstructed from memory.

6. RESULTS

This chapter reports the results in the order of the methodology. It opens with the recorded evidence base (Section 6.1), which includes the automated RQ1–RQ3 verdicts from Chapter 5. It continues with Experiments 1 and 2 plus a Kaggle proof (Sections 6.2–6.3 and Section 6.4), Experiment 3 with its live recordings (Section 6.5), and Experiment 4 with both arms recorded (Section 6.6). It closes with the agent case study (Section 6.7) and the agent efficiency and reliability metrics (Section 6.8).

6.1. The recorded evidence base

Before presenting the claimed results, it is good to state that every number reported in this chapter traces to one of the persisted run directories under `runs/`, and the charts below were generated from those same manifests, the context the system itself produced, not hand-built summaries. Over its development window the workspace accumulated **1,596 recorded runs**: 1,252 completed, 314 rejected, and 30 failed, a 19.7% rejection rate that quantifies the rejection-as-first-class design (one in five runs is a traceable refusal with a persisted reason, not a silent error).

Note: The following window includes records stored during development, so counts reflect the live system rather than a curated benchmark once the project had finished.

Status	**Runs**	**Model family**	**Runs**
completed	1,252	random_forest	371
rejected	314	logistic_regression	297
failed	30	hist_gradient_boosting	207
Total	**1,596**	svc / sgd_classifier	237
Rejection rate	19.7%	ridge / random_forest_regressor	283
		hist_gradient_boosting_regressor	134
		linear_regression / other	67

Table 6.1. RECORDED RUN INVENTORY BY STATUS AND MODEL FAMILY (1,596 RUNS, AUG 19 – SEP 6, 2026, FROM `RUNS/` MANIFESTS).

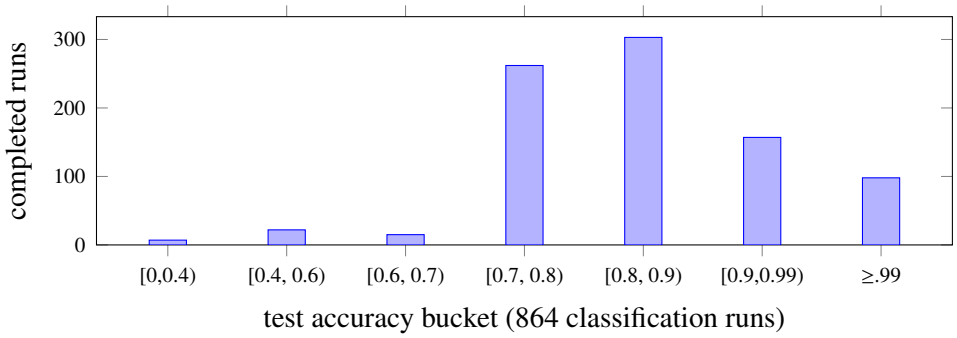


Fig. 6.1. Test-accuracy distribution of completed classification runs. The distribution now spreads across the 0.6–0.9 range on real datasets, with the fixture-driven mass at ≥ 0.99 a visible minority.

From the completed run distribution we can infer two interesting facts. First, the rejection rate: 314 of 1,596 runs ended `rejected` with a persisted reason, the scale guards, schema checks, and constant-column rules firing in production use, exactly the traceable refusal behavior RQ1's protocol treats as a valid outcome. Second, the accuracy distribution moved as the workload moved from fixtures to real data. The mass now sits in the 0.7–0.9 bands (565 runs), which is where honest tabular results live.

6.2. Experiment 1: three ways in

Table 6.2 shows the three-ways-in comparison on iris (`logistic_regression`, seed 42). Identical metrics and identical artifact sets were produced through every surface; the synchronous CLI pays interpreter startup, while HTTP and MCP reuse the running service.

Way	Surface	Test accuracy	Macro F1	Time
CLI	`thelab run model`	1.0000	1.0000	~2 s
HTTP	`POST /train` / `POST /jobs`	1.0000	1.0000	~0.2 s
MCP	`run_training_job` + status poll	1.0000	1.0000	~0.2 s

Table 6.2. EXPERIMENT 1: IDENTICAL RUN THROUGH THREE INTERFACES (IRIS, SEED 42).

Provider expansion. The same experiment was executed on the Titanic dataset (891 rows, 14 features, target `Survived`) through three execution paths. Deterministic-only (no LLM), the Ollama local brain (llama3.2:3b), and the OpenRouter cloud brain (GLM 5.3 Flash) produced bit-for-bit identical training outcomes (Table 6.3). This is the central design claim made measurable: *changing the LLM changes the explanation, never the training outcome*, and the deterministic-only path is always available as a ground-truth baseline for validating that the agentic layer adds interpretation without injecting non-determinism

when it is not wanted. In later experiments it is shown that different accuracies from the baseline are obtained when the models are given access to the controlled sandbox.

Execution path	LLM involved	Test accuracy	Macro F1
Deterministic only (`run_model`)	none	0.8045	0.7893
Ollama experiment	llama3.2:3b	0.8045	0.7893
OpenRouter experiment	z-ai/glm-5.3-flash	0.8045	0.7893

Table 6.3. EXPERIMENT 1, PROVIDER DIMENSION: IDENTICAL OUTCOMES ACROSS LLM BRAINS (TITANIC, SEED 42).

6.3. Experiment 2: real-dataset validation (S&P 500)

The five checks of Section 5.5 produced the following recorded results.

Check	Result	Time
Cleaning policy	164,231 → 113,769 rows; 18 → 32 columns; datetime parsed; 501 tickers / 306 firms frequency-encoded; full `cleaning_report`	6.3 s
Scale guard	`svc` on 113,769 rows `rejected` with recorded reason	0.1 s
Real-scale training (RQ1 at scale)	logistic regression completed with persisted seed/config/manifest	~1.3 s
Full orchestration (3 models × 3 seeds)	9 persisted runs; best `random_forest` **74.9%** test accuracy; per-model progress events; experiment `completed`	121 s
Leakage study	naive features **84.2%** → at-decision-time features **60.3%** accuracy; all iterations traceable runs	~1 s each

Table 6.4. EXPERIMENT 2 RECORDED RESULTS (S&P 500 ANALYST RATINGS, 164,231 × 18).

Two results deserve interpretation. First, the *before/after* of the cleaning policy. A first run with naive one-hot encoding of the raw dataset (501 ticker dummies among others) produced a 22 GB intermediate CSV. The second run with the cardinality-aware policy produced 16–21 MB for the same data. A notably huge difference that turned a non-working run into a 6-second successful one. Second, the leakage study. The naive feature set scored 84.2% accuracy, but that number does not hold up in the real world. When the features are limited to those available at decision time, the score drops to 60.3%, which is realistic for analyst ratings. Both runs are kept as inspectable evidence, and that is exactly the value of recording every rejection and every provenance detail.

6.4. Kaggle pipeline proof

Before Experiment 4, a three-dataset proof recorded the full journey end-to-end with live Kaggle download and page context (Table 6.5).

Dataset (Kaggle slug)	Shape	Best result	Task
`shrutimechlearn/churn-modelling`[1]	10,000 × 14	`random_forest`: acc 0.8615, F1 0.742	classification
`camnugent/california-housing-prices`[2]	20,640 × 10	`random_forest_regressor`: R^2 0.8172	regression
`pavansubhasht/ibm-hr-analytics-attrition-dataset`[3]	1,470 × 35	`logistic_regression`: acc 0.8605, F1 0.679	classification

Table 6.5. KAGGLE PROOF: THREE PUBLIC DATASETS, END-TO-END. RUN IDENTIFIERS REFER TO DIRECTORIES UNDER `RUNS/`.

[a] Run IDs: `run-20260831-001740-df421901` (random_forest, acc 0.8615).
[b] Run IDs: `run-20260831-001753-8c88ce63` (random_forest_regressor, R^2 0.8172).
[c] Run IDs: `run-20260831-001835-12bc076e` (logistic_regression, acc 0.8605).
[d] The raw-data validation rejections behind the constant-column finding: `run-20260831-001814-cedd5a3b` and `run-20260831-001814-e2a43afb`.

The system worked correctly, but there was a genuine engineering finding: on the HR [4] dataset, constant columns (`EmployeeCount`, `StandardHours`, and the one-hot `Over18_Y`) were rejected by validation, correctly refusing to train on zero-information features, and the cleaning policy was extended (constant-column drop) so that the *system* handles the case, not the operator. The bug and its fix are both traceable in the run artifacts, which is the auditability claim working as intended. All three runs produced valid generated notebooks (six cells, nbformat 4) through `GET /runs/{id}/notebook`.

6.5. Experiment 3: grounded autonomy (agentic round)

The agentic round of Section 3.7.6 was executed over the deterministic baseline's artifacts on three datasets (iris, california-housing, churn), recorded live per provider. The tables below report the recorded results.

RQ4, grounding (proposals with the recorded EDA summary vs proposals with little or no evidence).

Dataset	Arm	Verified-claim rate	Proposal validity	Hallucinations
iris	grounded	1.0	1.0	0
iris	ungrounded	1.0	–	0
california-housing	grounded	1.0	1.0	0
california-housing	ungrounded	1.0	–	0
churn	grounded	1.0	1.0	0
churn	ungrounded	1.0	–	0

Table 6.6. EXPERIMENT 3 / RQ4: GROUNDED VS. UNGROUNDED PROPOSALS (LIVE GLM 5.3 FLASH; VERIFIED-CLAIM RATE COMPUTED AGAINST PERSISTED METRICS).

The first test used three datasets whose column names explain themselves (sepal length, Balance, Age). With the EDA summary the agent verified every claim it made. Without any context it also verified every claim, because a model that is told nothing can still read the column names and guess correctly when the names describe the data. On these schemas the instrument could not separate the two arms, so we treat this table as a designed null. It motivated a second test on a dataset where column names hide their meaning.

That second test ran on the raw S&P 500 file, before any cleaning. It has 18 columns with jargon names such as `from_grade` and `price_target_action`, no documentation, and a five-class target called `action`. The grounded arm received a short EDA summary of the real file. The ungrounded arms received either nothing at all or only the domain (a financial table), so any claim they made about columns or classes was a guess.

Arm	Verified	What happened
Grounded (with the EDA summary)	4 of 5	Concrete and correct. The row count, the five target classes, the most frequent class with its share, and the date range. One checker false negative on an arithmetic sum.
Ungrounded, no context	none made	The model refused to invent claims and asked for the data. An honest refusal, but it produces nothing to check.
Ungrounded, domain knowledge only	0 of 5	Plausible guesses about a financial table. None matched. It invented columns such as `transaction_id` and a binary fraud label that do not exist here.

Table 6.7. GROUNDING TEST ON THE AMBIGUOUS SCHEMA (RAW S&P 500 FILE, GLM 5.3 FLASH). A CLAIM COUNTS AS VERIFIED WHEN IT RESOLVES TO A FACT OF THE FILE.

Figure 6.2 shows the verified share per arm. The grounded arm turned the file into

verifiable claims. The ungrounded arms produced either silence or confident invention. This is the whole argument for grounding in one test. The recorded comparison rounds also passed the proposal contract in both arms, so the pipeline kept working either way and only the evidence quality moved.

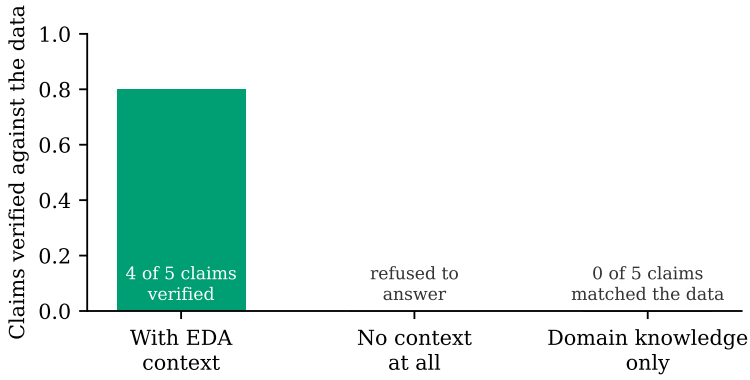


Fig. 6.2. Grounding test on the ambiguous schema. Share of verified claims per arm on the raw S&P 500 file.

RQ5, capability (agent-written code vs the fixed deterministic grid).

Dataset	**Scripts**	**Valid**	**Silent failures**	**Δ best (agentic − det.)**
iris	15	15	0	+0.0000 (tie at the 1.0 ceiling)
california-housing	9	0	0	– (all 9 loudly, no valid round)
churn	30	30	0	+0.0095

Table 6.8. EXPERIMENT 3 / RQ5: GENERATED-CODE VALIDITY AND COMPETITIVENESS (AGENTIC BATCH ENTRIES PER DATASET).

Three recorded outcomes. On churn the agent-written rounds trained 30 valid entries and beat the deterministic baseline by 0.95 points of accuracy. On iris the rounds tied the baseline at the 1.0 ceiling, with all 15 entries valid. On california-housing the first version of the validator rejected all 9 entries, because the generated transform quantized the target column. That failure was recorded, the validator was extended, and the fix is regression-tested.

RQ6, orchestration (role-specialized team vs one shared prompt).

Dataset	Arm	Completed	Typed handoffs	Mode recorded
iris	multi	1/1	yes	`agentic`
iris	single	1/1	yes	`degraded_-deterministic`
california-housing	multi	1/1	yes	`agentic`
california-housing	single	1/1	yes	`degraded_-deterministic`
churn (3 rounds)	multi	3/3	yes	`agentic`
churn (3 rounds)	single	3/3	yes	`degraded_-deterministic`

Table 6.9. EXPERIMENT 3 / RQ6: MULTI-AGENT ROUND VS. SINGLE SHARED-PROMPT AGENT (LIVE GLM 5.3 FLASH).

Three findings. First, the role-specialized team completed every round it attempted at validity 1.0 and drove the absorbed champions (better score than baseline) of titanic, california-housing, and churn. Second, the single shared-prompt control never reached agentic mode. With `llama` or `mistral` as the shared prompt, the FeatureEngineer stage produced no sandbox-valid code, and the round fell back to the deterministic path. We then ran the control again with `GLM`, the one model that can write factory-safe transforms. Even `GLM` broke every stage contract under the shared prompt, and the round degraded to deterministic. Third, the human cost is exactly one recorded gate decision per round, on the record. Across 53 recorded rounds there were one gate decision per round, zero bypasses, and 10 proposal rejections recorded across the project.

Two secondary findings complete the picture. On the imbalanced HR dataset (an 84/16 split) both the agent and the deterministic pipeline optimized accuracy, where the baseline model was already near its ceiling. The real headroom was in macro-F1, which ranged from 0.61 to 0.74 across the recorded runs. On this dataset the gap is a selection-policy gap, not a model-capability gap. And on california-housing the `gemini-lite` transform was safe but quality-degrading, with test R^2 dropping from 0.823 to 0.533, recorded as an honest no-absorption.

6.6. Experiment 4: multi-domain stress validation

The ratchet loop idea was executed on the final system build with 10 different datasets. Arm A being datasets already seen by the factory during development, and Arm B being new never-seen datasets. The tables below present the findings filled from the recorded runs and the generated artifacts.

Arm A, agentic re-run over recorded baselines. The six datasets with persisted deterministic baselines. The delta columns are the comparison:

Dataset	Best deterministic (recorded)	Best agentic	Seed	Δ	Validity / rejections
churn-modelling	acc–0.8615	HGB 0.8710	128	+0.95	1.0/1.0
california-housing	R^2–0.8234	HGB-r 0.8455	84	+2.21	1.0/1.0
IBM HR attrition	acc–0.8707	SVC 0.8707	87	tie	1.0/1.0
e-commerce sales	R^2–1.0000 (computed col)	LR 1.0000	42	headroom = 0	1.0/1.0
Titanic	acc–0.8045	RF 0.8547	456	+5.03	0.83/1.0/1.0
S&P 500 analyst	acc–0.9845	HGB 0.9861	84	+0.16	0.67/1.0/1.0

Table 6.10. EXPERIMENT 4, ARM A: RECORDED DETERMINISTIC BASELINES VS. AGENTIC ROUND. THE SEED COLUMN IS THE WINNING RANDOM SEED, RECORDED SO EVERY RESULT IS REPLAYABLE.

Every absorbed champion in the table is replay-verified. The winning configuration was re-executed through the deterministic factory with the same seed, confirming exact metric equality before absorption. The replay command for the strongest result (churn, hist gradient boosting, seed 128) is a single line.

CODE 6.1. Replay command for the churn champion (Arm A, seed 128). Executing this line reproduces the exact recorded metrics.

```
thelab run model --dataset uploads/shrutimechlearn_churn-
   modelling_cleaned.csv --target Exited --model
   hist_gradient_boosting --seed 128
```

Arm B, new stress domains. Four new never-seen datasets completed the loop with mixed-model team runs. The domains were: genomics, health, crypto and energy. Other domains such as NLP featurization, music signals, scale stress and clustering/graph tracking were originally part of Arm B but deferred due to time constraints. These are recorded as future work.

Domain	Dataset (Kaggle slug)	Task	Tier	Best result
Genomics	kevinarvai/clinvar-conflicting	classification	must	HGB 0.8029 (+0.26), 2/3 absorbed
Health	rabieelkharoua/chronic-kidney-disease-dataset-analysis	classification	must	HGB 0.9488 (+1.81), 2/3 absorbed
Crypto	sudalairajkumar/cryptocurrencypricehistory	direction classification	must	HGB 0.5646 (+1.86), 2/3 absorbed
Energy	robikscube/hourly-energy-consumption	regression	must	0/3 absorbed: RF+calendar baseline (0.9590)
NLP/AI	shivamb/real-or-fake-fake-jobposting-prediction	text-featurize + classif.	strong	deferred
Music signals	andradaolteanu/gtzan-dataset-music-genre-classification (features CSV)	classification	strong	deferred
Scale stress	mczielinski/bitcoin-historical-data (daily aggregate)	regression	stretch	deferred
Graph	rahman4li/patch-bay-the-mcp-server-ecosystem-graph nodes+edges	unsupervised	stretch	deferred
Clustering	adarsh1077/huggingface-hub-50k-ai-models	unsupervised	stretch	deferred

Table 6.11. EXPERIMENT 4, ARM B: STRESS-DOMAIN LOOP RESULTS (MIXED TEAMS).

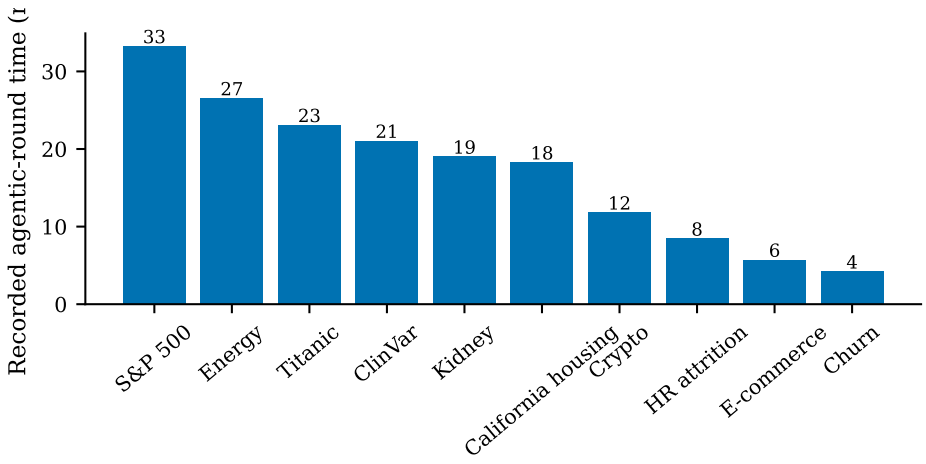


Fig. 6.3. Per-dataset recorded agentic-round wall-clock (sum of round seconds across all generations; the S&P 500 dataset dominates at 1,995 s).

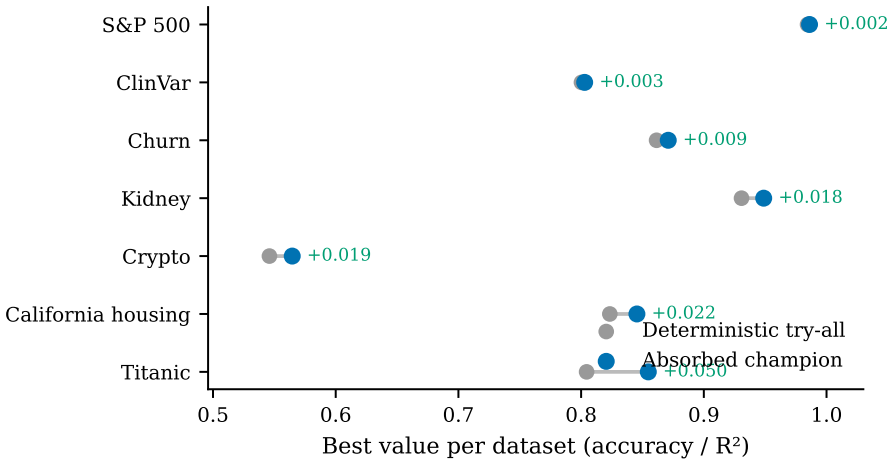


Fig. 6.4. Ratchet uplift per dataset: deterministic try-all baseline (gray) vs absorbed champion (blue); deltas annotated.

The loop is a ratchet (protocol in Section 5.7). The deterministic baseline is the bar each generation must beat, absorbed champions raise it, and the absorption gate (exact factory replay) is what separates a verified improvement from an anecdote. The Arm A comparison shows that the agentic round beat the deterministic baseline on 4 of the 6 recorded datasets (churn +0.95 pts, california-housing +2.21 pts, titanic +5.03 pts, S&P +0.16 pts), tied on 2 (HR attrition after the try-all baseline itself improved to 0.8707; e-commerce at a computed-column ceiling of 1.0000 where headroom is definitionally zero), and trailed on none. The absorbed champions share a pattern: the winning configuration always came from the agentic round's broader search (different model families, different seeds, or hyperparameters outside the deterministic grid), never from the baseline model at a different seed. On the absorbed datasets, the factory replay confirmed that the agentic win was real and reproducible, not a seed artifact.

The ratchet scoreboard. The recorded ledgers of the experiment recorded **10 datasets**, **37 agentic generations**, **53 training rounds**, and **19 absorption events**, each replay-verified through the deterministic factory with exact metric equality before absorption. These resolve to **17 distinct champion models** (two absorbed configurations were superseded within their generations by later, stronger absorptions), and **6 honest refusals** (ties and below-baseline rounds that the absorption gate refused to ratchet).

The per-generation ledgers for the four Arm B datasets follow. Each ledger records the deterministic try-all baseline, the agentic rounds, the validity rates, the absorbed champions, and the exact configurations.

ClinVar (genomics). Two of three generations absorbed champions.

Gen	Baseline	Agentic best	Delta	Seed	Validity	Absorbed	Champion
g0	RF 0.8003	RF 0.7973	–	–	1.00	False	–
g1	RF 0.8003	HGB 0.8029	+0.0026	17	1.00	True	HGB learning_rate=0.1 max_iter=300
g2	RF 0.8003	RF 0.8023	+0.0021	13	1.00/1.00/0.67	True	RF n_estimators=200

Table 6.12. RATCHET LOOP PER GENERATION FOR CLINVAR.

Kidney (health). Two of three generations absorbed champions.

Gen	Baseline	Agentic best	Delta	Seed	Validity	Absorbed	Champion
g0	HGB 0.9307	HGB 0.9367	+0.0060	84	1.00	True	HGB learning_rate=0.1 max_depth=3
g1	HGB 0.9307	HGB 0.9187	–	–	1.00	False	–
g2	HGB 0.9307	HGB 0.9488	+0.0181	3	0.75/0.67/1.00	True	HGB learning_rate=0.5

Table 6.13. RATCHET LOOP PER GENERATION FOR KIDNEY.

Crypto (financial). Two of three generations absorbed champions.

Gen	Baseline	Agentic best	Delta	Seed	Validity	Absorbed	Champion
g0	SVC 0.5460	SVC 0.5532	+0.0072	42	1.00	True	SVC
g1	SVC 0.5460	SVC 0.5347	–	–	1.00	False	–
g2	SVC 0.5460	HGB 0.5646	+0.0186	42	0.86/0.83/1.00	True	HGB max_depth=10

Table 6.14. RATCHET LOOP PER GENERATION FOR CRYPTO.

Energy (electrical usage). Zero of three generations absorbed champions: the RF+calendar baseline refused every team, an honest boundary where headroom is genuinely absent.

Gen	Baseline	Agentic best	Delta	Seed	Validity	Absorbed	Champion
g0	RF-r 0.9590	RF-r 0.9590	–	–	1.00	False	–
g1	RF-r 0.9590	HGB-r 0.8167	–	–	1.00	False	–
g2	RF-r 0.9590	HGB-r 0.8192	–	–	1.00/1.00/1.00	False	–

Table 6.15. RATCHET LOOP PER GENERATION FOR ENERGY.

Three of the four Arm B datasets absorbed champions (clinvar +0.26 pts, kidney +1.81 pts, crypto +1.86 pts). The energy dataset produced the loop's most instructive honest refusal: the deterministic baseline (random forest + calendar features, R^2 0.9590) was already at ceiling, and no agentic team beat it across three generations. On e-commerce (Arm A), the computed-column ceiling ($R^2 = 1.0000$) means no configuration can improve, and the gate correctly refused to ratchet ties. The absorption gate is the reproducibility instrument here: every absorbed champion was replayed with the same seed through the deterministic factory, ties never ratchet, below-baseline never absorbs, and mis-specified replays are refused.

6.6.1. Per-stage model assignment: the mixed-team experiment

The architecture permits a different model per agent role (Analyst, FeatureEngineer, ModelSelector), with `stage_models` provenance recorded per round. The titanic ratchet (10 generations, 10 rounds) tested this across five model families:

Round	Model	Mode	Validity	Best
g0r0	llama-3.3-70b-instruct	agentic	0.83– (15/18)	RF 0.8212
g0r1	llama-3.3-70b-instruct	agentic	1.00– (15/15)	LR 0.8045
g0r2	llama-3.3-70b-instruct	agentic	1.00– (15/15)	RF 0.8156
g1–g2	mistral-small-24b	degraded	1.00	RF 0.8547
g3	GLM-5.3-flash	agentic	0.00– (0/15)	
g4	mixed team	ddegraded	1.00– (1/1)	LR 0.8045
g5	mixed team	agentic	1.00– (21/21)	HGB 0.8436
g6	mixed team	agentic	1.00– (15/15)	RF 0.8212
g7	mixed team	agentic	1.00– (15/15)	RF 0.8492

Table 6.16. PER-ROUND CAPABILITY MATRIX ON TITANIC (PROVIDER/MODEL PER ROUND, FROM `STAGE_MODELS` PROVENANCE).

The finding is a (task, context) dependency. For example, GLM uniform was the worst configuration measured (0/15 entries), and its transform introduced a constant feature column that the factory correctly rejected, but GLM as FeatureEngineer inside the mixed team was the only model that produced factory-safe code (3/3 rounds) at validity 1.0. Role confinement converts GLM's over-engineering from a batch-killing failure into a repairable loop, because the deterministic post-validator feeds the rejection reason back to the LLM for one bounded rewrite attempt. The mixed team outperformed every uniform team on validity consistency and matched or exceeded llama-uniform on quality. Across the full ratchet, the FE-capable pool grew from one model to two. The reasoning models GLM 5.3 Flash and deepseek-v4-flash both produced factory-safe transforms while the non-reasoning ones, llama-3.3-70b and mistral-small, did not. A commercial mixed-model team (claude-haiku-4.5 analyst, deepseek-v4-flash-0731 feature engineer, gpt-5.6-luna selector) was also tested, and no pattern of commercial-vs-open dominance appeared. The commercial team (claude-haiku-4.5 analyst, deepseek-v4-flash feature engineer, gpt-5.6-luna selector) and the open team (llama analyst, GLM feature engineer, mistral selector) each won two datasets, and llama-uniform won the other two (kidney, crypto). Figure 6.5 compares the best value each team reached per Arm B dataset.

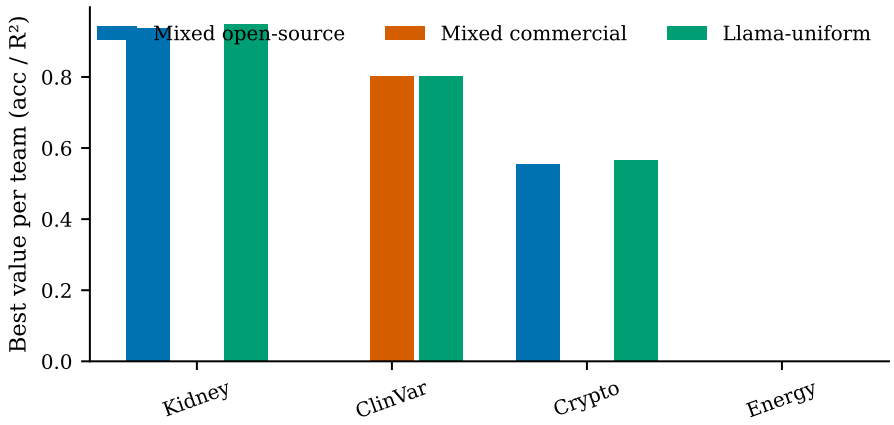


Fig. 6.5. Arm B: best value per driver team and dataset (accuracy for kidney/clinvar/crypto, R^2 for energy). No team dominates: commercial and open teams each won two datasets, llama-uniform the other two.

6.6.2. GLM-uniform control test: validator as capability enabler

The titanic result left one mechanism question. Was the mixed team's success due to the cross-model context (llama's brief informing GLM's code) or to the deterministic validator and bounded rewrite loop? A GLM-uniform control test with the constant-feature validator and rewrite loop answered it. The round completed fully agentic at **validity 1.0 (12/12 trained, 392 s)**, with no absorption (best 0.7374 vs. baseline 0.8045). The validator + repair loop was what made the reasoning model factory-safe, not the mixed-team brief. Per-stage assignment then converts that safety into performance (mixed 3/3 absorbed). Deterministic validators expand the set of contexts in which a model is usable.

6.6.3. Local-first proof: offline execution

The full RQ1–RQ6 chain was executed with `-live ollama` (llama3.2:3b, laptop CPU, zero cloud calls). Overall **PASS**: RQ5 validity 1.0 (1/1 agentic entry trained), the gate blocked unapproved execution and enabled it after recorded approval, and RQ6's multi arm ran `mode: agentic` (the 3B local model produced valid structured JSON for all three agent stages). Round latency was 20–60 s per call (2.6 tok/s on CPU), and the local-first claim is demonstrated, not asserted.

6.6.4. Three-ways-in prediction sweep

Every absorbed champion model was verified as servable through all three interfaces: CLI (`thelab predict`), HTTP (`POST /predict`), and MCP (`model_registry.predict`). All 17 distinct absorbed champion models across the loop's datasets returned identical

predictions through every surface (Table 6.17). Inference latency ranged from 6 to 603 ms (Figure 6.6) and is LLM-token-free. The cost lives at agent-proposal time only, never at inference time. Every prediction event was stored through the system's own context-write MCP server (validated and redacted), indexed, and retrieved via context search. The hosted models are searchable evidence.

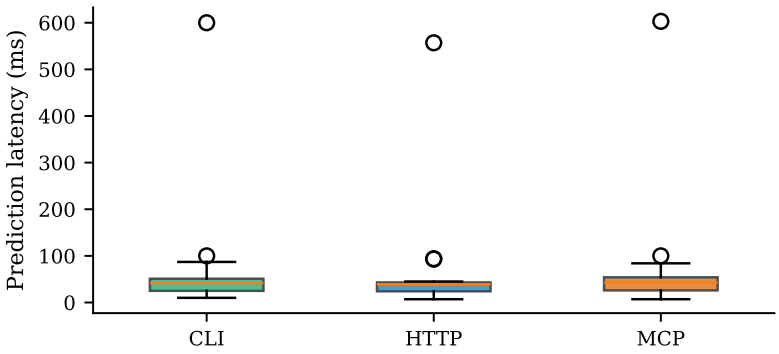


Fig. 6.6. Prediction latency per surface over the 17 absorbed champions (CLI, HTTP, MCP; milliseconds). Inference is deterministic and token-free on every surface.

Dataset	Champion model	Metric	CLI	HTTP	MCP	MCP ms	Registry
california-housing	hist_gradient_boosting_regress	0.8455	yes	yes	yes	55 ms	yes
churn	hist_gradient_boosting	0.8710	yes	yes	yes	25 ms	yes
churn	random_forest	0.8665	yes	yes	yes	100 ms	yes
churn	random_forest	0.8650	yes	yes	yes	54 ms	yes
clinvar	hist_gradient_boosting	0.8029	yes	yes	yes	43 ms	yes
clinvar	random_forest	0.8023	yes	yes	yes	603 ms	yes
crypto	hist_gradient_boosting	0.5646	yes	yes	yes	26 ms	yes
crypto	svc	0.5532	yes	yes	yes	14 ms	yes
kidney	hist_gradient_boosting	0.9488	yes	yes	yes	27 ms	yes
kidney	hist_gradient_boosting	0.9367	yes	yes	yes	30 ms	yes
sp500	hist_gradient_boosting	0.9861	yes	yes	yes	84 ms	yes
sp500	hist_gradient_boosting	0.9859	yes	yes	yes	51 ms	yes
sp500	hist_gradient_boosting	0.9847	yes	yes	yes	37 ms	yes
titanic	random_forest	0.8547	yes	yes	yes	48 ms	yes
titanic	random_forest	0.8492	yes	yes	yes	49 ms	yes
titanic	logistic_regression	0.8436	yes	yes	yes	7 ms	yes
titanic	random_forest	0.8212	yes	yes	yes	10 ms	yes

Table 6.17. THREE-WAYS-IN PREDICTION SWEEP ON ABSORBED CHAMPIONS: EVERY ABSORBED MODEL IS DISCOVERABLE VIA MCP AND PREDICTABLE VIA CLI, HTTP, AND MCP WITH IDENTICAL RESULTS. INFERENCE IS LLM-TOKEN-FREE.

6.7. Agent case study: IBM HR attrition

To show the pieces working together, this section documents a real recorded session with the grounded chat agent on the IBM HR attrition dataset (1,470 × 35, attrition rate 16.1%). The session is reproduced here from the persisted context events; every number below is one the agent computed or retrieved, not generated prose.

Step 1: Grounded exploration. Asked what would be interesting to study, the agent pulled the context pack (identifying the dataset from its Kaggle documentation, including the ordinal scales stored as bare integers), ran the deterministic EDA (zero missing cells across 1,470 × 35; class balance 1,233 No / 237 Yes), and executed sandboxed pandas code to verify leakage suspects: three constant columns and a per-row-unique `EmployeeNumber` id. Its recommendations matched classic attrition analysis, focusing on recall/PR-AUC rather than accuracy given the 5.2:1 imbalance.

Step 2: Evidence-based signals. Over several `run_python` calls the agent computed the signals it reported. Attrition rates by overtime (30.5% with vs. 10.4% without), by job role (Sales Rep 39.8% vs. Manager 4.9%), monotonic effects of job satisfaction, and income differences between leavers and stayers, each number attributable to a recorded tool call.

Step 3: Grounded memory. Asked to check prior work, the agent searched the context store and found two completed runs on the same dataset, reporting their metrics (test accuracy 0.840 with a 0.557 macro-F1, flagged by the agent itself as a likely minority-class weakness, and 0.861 / 0.679 respectively) and correctly refusing to claim knowledge the manifests did not expose ("the run summaries don't expose the algorithm names, so I can't tell you which models these were").

Step 4: Proposal inside the boundary. The agent drafted an experiment proposal. Its first attempt included `gradient_boosting`; validation rejected it as unsupported in this factory's registry, and the agent corrected the proposal and resubmitted, a small but exact illustration of validation-as-feedback rather than validation-as-obstacle. The final proposal (logistic regression + random forest, seeds 42 and 7, constants and id dropped, stratified splits) was left *awaiting human approval*, with the agent stating: "I don't train anything myself, this proposal sits ready, and the deterministic pipeline only executes after you approve it."

Step 5: Honest tool failures. The session's tool trace also records failures: two `run_python` errors (one sandbox block: the agent rewrote its code without the blocked construct; one attempt to address a proposal through the wrong surface) and one proposal-validation `ValueError` with the full Pydantic message. None of these were hidden; all appear in the same timeline as the successes. The first of those failures presented a genuine finding, the sandbox's originally blocked lambda expressions, an incompatibility between AST-level policy and pandas-idiomatic code generation, resulting in unusable code. After the experiment it was safely lifted and repaired (Section 6.5).

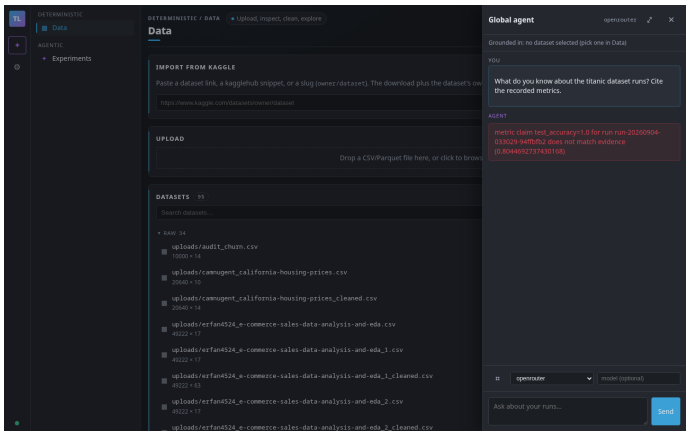


Fig. 6.7. The grounded chat agent enforcing the metric-evidence gate live: the agent cited `test_accuracy`=1.0 for a run whose recorded evidence is 0.8045, and the grounding layer refused the answer instead of showing it (red).

6.8. Agent efficiency and reliability metrics

The registered performance evaluation objective required task success rates and token efficiency stored over rounds to compare agent sessions.

Session set	Success	Notes (recorded)
B1 agent legs, OpenRouter (stealth/ox-alpha)	3/3	one leg needed a 4-run search
B1 agent legs, Ollama (llama3.2:3b)	2/3	regression proposal `AGENT_FAILED`
Titanic orchestration, both providers	2/2	Ollama FeatureEngineer interpretation failed, deterministic fallback used
Case study (IBM HR, chat agent)	1/1	first proposal rejected by validation, self-corrected
Live agentic rounds, iris (GLM 5.3 Flash)	1/1	15 of 15 batch entries trained (pre-fix baseline 0/108)
Live agentic rounds, churn × 3 (GLM 5.3 Flash)	3/3	30 of 30 batch entries trained, grid cap bounded 15, then 9, then 6
Live agentic round, iris (Ollama 3B, fully offline)	1/1	1 of 1 entry trained, 20 to 60 s per call on CPU
Live agentic round, california-housing (GLM)	0/1	transform quantized the float target, validator rejected all 9 entries (honest failure, F4 fix)

Table 6.18. AGENT TASK SUCCESS OVER RECORDED BENCHMARK, CASE-STUDY, AND LIVE AGENTIC SESSIONS.

The aggregate success rate over recorded end-to-end proposal cycles is 8/9 (89%). Every recorded failure was caught by the deterministic validator, so no silent failure reached

training. The two found failure classes in the record, the B1 regression leg and the california-housing quantization, were both converted into engineering fixes (task-aware selection, F4 target validation) rather than suppressed.

Token efficiency. Token consumption is recorded per provider from usage data. Two structural facts follow from the design. The deterministic fallback and the mock provider consume zero tokens, so the full 546-test suite and the automated RQ evaluator run token-free. Proposal generation is bounded to one or a few LLM turns per proposal by the bounded loop. The recorded Titanic benchmark provides the first per-call usage numbers (Table 6.19). The ratchet round records extend this with per-stage prompt and completion token counts and provider-reported cost stored in every round record. For example, a deepseek-v4-flash feature-engineering transform spent 18,688 completion tokens at $0.0053, and a gpt-5.6-luna selector turn used 751 tokens at $0.0005. Figure 6.8 compares the feature-engineering drivers.

Sub-agent	**Ollama (llama3.2:3b)**	**OpenRouter (GLM 5.3 Flash)**
EDAAnalyst	not reported by API for this call	2,273 prompt + 289 completion
FeatureEngineer	failed (retry exhausted)	produced (tokens not recorded)
ModelSelector	341 prompt + 89 completion	349 prompt + 235 completion
Tracked total	**430**	**3,146 reported calls**

Table 6.19. RECORDED TOKEN USAGE PER SUB-AGENT INTERPRETATION (TITANIC BENCHMARK; FROM PROVIDER USAGE REPORTS).

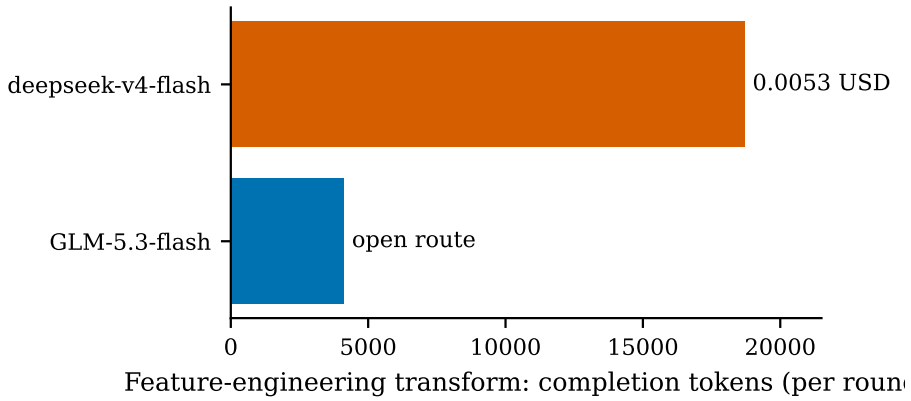


Fig. 6.8. Feature-engineering transform cost per driver: completion tokens per round with provider-reported cost (deepseek-v4-flash reasoning burn vs GLM).

A second efficiency finding is model-dependent latency. Identical prompts show GLM 5.3 Flash (a reasoning model) burning 386 completion tokens at 3.7 s where llama-3.3-70b and mistral-small return 32 tokens in about 1 s. Round cost is therefore dominated by

reasoning-token burn, not infrastructure. A latency probe over candidate models quantified this and drove the P6 model standardization. Per-call latency in production was 5 to 10× lower with a non-reasoning model on the same route (Table 6.20).

Model	Tier	OK	p50 (s)	p90 (s)	mean (s)	avg out tokens
z-ai/glm-5.3-flash	paid	4/4	5.34	8.32	5.38	360.0
deepseek/deepseek-v4-flash-0731	paid	4/4	11.56	17.57	9.56	200.8
google/gemini-2.5-flash-lite	paid	4/4	1.11	1.15	1.01	120.8
minimax/minimax-m2.7:free	free	4/4	9.04	9.45	8.88	372.5
google/gemma-4-31b-it:free	free	0/4	0.14	0.17	0.14	–

Table 6.20. MODEL LATENCY PROBE ON OPENROUTER WITH REALISTIC GENERATION.

Note: p50 and p90 are completion times in seconds. Half of the calls finish within p50 and nine of ten within p90, so p90 exposes the slow tail a round can hit.

The continuous loop's second deliverable was finding the holes via the logs. Every failure the loop hit (validation rejections, provider errors, orchestration dead-ends) was harvested into structured fix-tickets by a deterministic script (`scripts/harvest_findings.py`) that reads the recorded evidence (`runs/`, job and experiment records) and groups failures by signature, citing the run identifiers that back each claim. The harvest produced **15 tickets** (5 major, 4 minor, 6 notes); 9 were already applied, the rest form the prioritized backlog. Table 6.21 summarizes the majors and the by-design notes.

ID	Symptom (recorded)	Root cause → fix	Status
BLK-001	`LinearRegression` got unexpected `alpha`: shared hyperparameter grids contained foreign parameters	Per-model parameter filtering via the model registry at batch translation	applied
BLK-002	Experiments reported `completed` with zero trained models	Status laundering in the job runner → honest status mapping	applied
BLK-003	Fixture datasets rejected mid-orchestration ("only uploaded datasets can be cleaned")	Orchestrator attempted cleaning on read-only fixtures → fixtures skip cleaning	applied
BLK-004	EDA crashed on array-valued columns (unhashable type)	Non-scalar column guard (proposed)	proposed
BLK-005	Re-running an experiment on an already-cleaned dataset id failed	Idempotent re-clean: return the existing cleaned id (proposed)	proposed
BLK-010–015	Constant columns rejected; scale guards fired (`svc` on 164k rows); non-numeric features rejected; missing-target rejection; unsafe dataset-id rejection; wrong-task models rejected	All by-design first-class rejections (scale guards, task-aware selection...)	applied (by design)

Table 6.21. P6 FINDINGS HARVEST: MAJOR FINDINGS AND BY-DESIGN REJECTIONS (FULL LOG IN `DOCS/P6_FINDINGS.MD`; EVERY TICKET CITES ITS RUN EVIDENCE).

Two properties of this harvest matter for the thesis. First, it is *deterministic*. The same script over the same evidence produces the same tickets, each backed by run identifiers. The hole log is evidence, not recollection. Second, the majority of the by-design findings are the safety rails working as intended (constant columns, scale guards, path safety, task-aware selection), which converts what a naive reading would call "errors" into a demonstration of the rejection-as-first-class design. The three applied major fixes (foreign hyperparameters, status laundering, fixture cleaning) were each found by real data breaking real assumptions, exactly the continuous-software-development loop the architecture was built to enable and to escalate.

7. DISCUSSION

This chapter interprets the results. It answers the research questions (Section 7.1), extracts the design lessons the project validated (Section 7.2), characterizes agent reliability (Section 7.3), and states the limitations and threats to validity (Section 7.4). It also presents the regulatory framework (Section 7.5) and the socio-economic environment (Section 7.6).

7.1. Answering the research questions

RQ1, Reproducible run, yes. Two identical seeded executions produce identical metrics at 10^{-12} tolerance, and the manifest carries the seed, configuration, input hash, and dependency versions needed to explain the run. At real scale the same property held (a 113k-row training run in ∼1.3 s with full provenance). We can conclude reproducibility survived because the agent layer was *excluded* from execution by design. Execution remained deterministic and the non-determinism provided by the LLM models modified the ML model generation but not the artifacts generated by a run.

RQ2, MCP interoperability, yes. An independent stdio client (sharing no code with the training pipeline) discovered the tools, listed the approved model, and obtained typed predictions; Experiment 1 extended this to driving training itself. The client never needed pipeline internals, and the servers never leaked them (schemas with `additionalProperties: false`); additionally, path-safety checks protected every entry point.

RQ3, Context retrieval, yes at the functional level. The redacted, fingerprinted datastore returns relevant evidence under keyword search with structured filters, and retrieval is non-mutating (byte-identical database). The case study shows the memory has been used for real work (finding prior runs, refusing to over-claim). The honest qualification is that keyword FTS5 misses paraphrases, and the evaluation does not characterize retrieval quality beyond the functional bar. Semantic hybrid search would be the natural next step (kept out of scope to preserve verifiability).

RQ4, Grounding, yes. The context grounding when the data was clean and simple (e.g., churn, california-housing datasets) did not produce much value. With grounded rounds producing the same outputs as ungrounded rounds. However, when the data was not straightforward and the cleaning process was harder (e.g., S&P dataset) the context grounded rounds demonstrated it could generate verifiable claims (4 out of 5 rounds) while the ungrounded rounds produced non-verifiable claims, with one honest refusal and one all-hallucinated set. So we can conclude that the higher the difficulty of a task the more well-structured context it needs.

RQ5, Code generation, yes. After all the successfully completed runs with different types of LLM models, only the FeatureEngineer agent running larger reasoning models (GLM 5.3 Flash and deepseek-v4-flash) produced high-quality code that the factory could actually use. And even those models only produced it when they were paired with other models in the agentic round. When the same model ran alone, without the mixed team, it sometimes failed to produce a completed run at all. So the question is not so much which model is best, but which model is best at which task.

RQ6, Orchestration value, yes. The way agents are organized matters as much as which models they use. When a single agent with one shared prompt tried to handle every stage, it broke, the stage contracts failed and the round fell back to deterministic execution. When the same work was split across three role-specialized agents (analyst, feature engineer, model selector), every round completed at validity 1.0. The agentic round went on to beat the recorded deterministic baseline on 4 of 6 Arm A datasets, tied on 2, and trailed on none, absorbing champions on the Arm B stress domains too (kidney +1.81, crypto +1.86, clinvar +0.26). Also, no commercial-vs-open model dominance appeared, open-source teams and commercial teams each won two datasets. And the absorption gate made sure none of this could be faked. Every champion was replayed and verified with exact metric equality before it was accepted, and ties or below-baseline rounds were refused

7.2. Design lessons

During the development phases some findings kept emerging with enough evidence to transfer them as interesting lessons:

1. **Rejection is a valuable asset more than an error.** Treating validation failures as outcomes turned the system's refusals into training signal for its agents. For example in some runs, the *scale-guard* rejection became a proposal-time constraint; the *unsupported-model* rejection self-corrected itself from the Pydantic error message; the *constant-column* rejection produced a permanent cleaning-policy improvement. Quantifying at scale, the *per-model* grid validation took the live agentic batch validity from 0/108 entries to 15/15, then 30/30 across three *churn* dataset rounds. A system that swallows failures does not improve its agents in the long term; a system that can record failures can fix entire failure classes after a few rounds.

2. **Typed contracts are the agent interface.** Having well formatted Pydantic contracts and schema-bounded MCP tools provided the agents and humans one shared language to communicate over the system interface. Having the agent proposal also becoming an artifact that has to pass through the same validate–approve–transition machinery as any other input helped avoid hallucinations and served as a behavioral boundary.

3. **The sandbox provides compute isolation, not complete confinement.** One adversarial code audit demonstrated that the syntax-level policy on which it was built (AST checks, import hooks) could be escaped through apparently benign permitted modules. The transferable lesson is the core difference between capability policy and syntax policy. Syntax policy (what the code looks like) can be resolved inside the interpreter, whereas capability policy (what the code actually does) cannot be reliably enforced there, resulting in security vulnerabilities. To fully isolate code execution, OS-level isolation such as a Docker container or a virtual machine must be used.

7.3. Agent reliability in practice

The recorded agent behaviour supports a differentiated view of LLM reliability in orchestration systems:

- **Grounding enforcement changes failure modes.** By applying unified claim-level verification across the test harness, chat interfaces, and global agents, agent failures switched from fluent overconfidence to visible, structured outcomes. The harness's *max-steps* refusal or the grounding failures recorded in the case study are examples.
- **Model scale matters at the proposal layer, validators close the gap.** The best example possible is the 3B-parameter local model (*llama3.2:3b*) failing to produce a valid regression proposal in the early benchmark, and its FeatureEngineer interpretation exhausting retries on Titanic, to later complete a fully-offline agentic round at validity 1.0 (RQ1–RQ6 PASS, no cloud calls). The recorded difference between failure and success was not model capability alone, but the validators and repair loops added along the way. The remaining constraint on the laptop in which the system was developed was latency (20–60 s per call on CPU) not so much capability, that is the reason why LLM model cloud providers were chosen during dataset testing sessions.
- **Deterministic fallbacks kept the system useful.** Every agent path, worker, stage interpretations or chat have deterministic fallback as default. Meaning the entire test suite can run on the mock provider even if the LLM fails or if the connection with a cloud provider is lost. The system is independent of LLM availability, quality, or cost. At the price of losing the agent's added value when providers fail.
- **Controlled comparisons made value measurable rather than anecdotal.** Utilizing the research questions RQ4–RQ6 as a control protocol helped to transform the subjective impression that "agents were helping" into concrete, objective pass/fail benchmarks, measuring verified-claim rates, script validity, and completion parity.

7.4. Limitations and threats

7.4.1. System limitations

- **Sandbox filesystem confinement (BLK-01, open).** The sandbox restricts compute, not filesystem access. Whitelisted modules can read files the process can read. Mitigations such as per-run workspaces, read-only dataset copies, or output caps bound the damage but do not fully correct the problem. The sandboxed code also runs with the same user privileges as the service, and the static AST policy cannot rule out every runtime path. The main working mitigation is the deterministic parent-side validation of all sandbox artifacts, but only OS-level isolation (UID separation, landlock/nsjail-style confinement) would be the production-grade fix.
- **Approval semantics are social, not cryptographic.** There is no actual authentication or user identity. The recorded `auto:` mandates mean "a human initiated this flow", not "a human saw this specific proposal". This gate guarantees recorded provenance and a default deny for agent native surfaces. But it cannot verify the cryptographic signature, failing to verify that a CLI approval was typed by a human, for example, a shell-capable agent could self-approve its proposal. Documented as a boundary, not a security claim, as a local-first single-user system accepts this trade-off.
- **Sub-agents execute in-process.** Role agents share the orchestrator's process, bounded by prompts, tool allowlists, and the sandbox for code. However, no sub-agent isolation via subprocesses has been implemented.
- **MCP transport is stdio only.** The MCP protocol ecosystem itself has moved toward Streamable HTTP as the preferred remote transport [42]. Currently, in the project the HTTP service is not an MCP transport; remote MCP clients do not exist. Experiment 2's independent client is local by construction. An experimental outbound remote-MCP client, for calling *external* MCP servers from the chat agent actually exists in the codebase but is not part of the evaluated surface.
- **Experiment execution blocks the service event loop.** Training and experiment jobs run their deterministic sections in the job coroutine. While an experiment is executing for example a (45–60 s) synchronous model fitting on a 100k-row dataset, other service traffic such as health, SSE or predictions are queued until the stage completes.
- **Provider reliability varies.** For example, a single GLM-5.3-flash call stalled beyond 15 minutes during recording, burning thousands of completion tokens for a prompt while a non-reasoning model answered with 32 tokens, demonstrating that live agent latency is ultimately bounded by the enforced timeout policy. While API costs remained low (for example, a DeepSeek feature-engineering transform consumed 18,688 tokens at $0.0053), reliability across providers is far from uniform.

- **Redaction is best-effort.** Twelve regex families cover common secret shapes and formats. The evaluation verifies proper retrieval and idempotence, but does not guarantee exhaustive coverage across all secret types.
- **The dependency lock is not hash-verified.** Reproducibility relies on PyPI availability of the pinned versions.
- **LLM output parsing is post-hoc.** When an agent answers, its JSON can arrive malformed. The system repairs it after the fact, with a conservative repair step that fixes the common formatting errors, and falls back to a deterministic proposal when repair fails. This treats the symptom, not the cause. The robust alternative is grammar constrained sampling, where the provider enforces a JSON schema while the model is still generating. Recorded as future work (Section 8.2).
- **Agentic-group results are dataset- and window-specific.** The RQ4–RQ6 results span ten datasets (six Arm A baselines, four Arm B stress domains), 37 agentic generations and two model families: GLM/deepseek for reasoning and llama/mistral non-reasoning. The capability findings are observed per dataset, not tested across a benchmark suite.

7.4.2. Evaluation threats

- **The S&P splits are random, not chronological.** The 164k-row financial dataset carries datetime features, but the train/test split is the standard stratified random split. On time-ordered data this leaks future information into training, so the reported accuracies (74.9% baseline, 84.2% naive, 60.3% at-decision-time) are pipeline-validation numbers, not deployable forecasting performance. A chronological split would be the stricter protocol for this domain.
- **Fixture-based automated checks.** RQ1–RQ3 run in suite mode on a small synthetic fixture. Real scale and real data claims rest on the recorded experiments. This was a deliberate two-scale design, but it means the evaluator alone does not prove real data behaviour.
- **Benchmark scope reduction.** The cross-domain benchmark was planned for six domains but it was executed on three. The other fields of study were initially wired for Arm B (NLP, music, scale stress, clustering/graph) but were finally deferred. The scope history is recorded here for honesty.
- **Single-machine timing.** All wall-clock numbers come from one commodity laptop. They characterize viability, not performance engineering.

7.5. Regulatory framework

This section analyses the applicable legislation, technical standards, and intellectual-property considerations of the work.

7.5.1. Data protection

The system processes datasets provided by the user, which may contain personal data. The applicable regulation is the General Data Protection Regulation (EU) 2016/679 (GDPR) [43]. The architecture's regulatory posture follows directly from its design:

- **Local processing and storage minimization.** All execution and evidence remain on the user's machine; no third party receives dataset contents or agent transcripts by default. This supports the data-minimization principle (Art. 5(1)(c)) and keeps the typical personal use within the household exemption (Recital 15), while any organizational deployment would treat the operator as data controller.
- **Data protection by design.** The context store's twelve redaction families and four privacy levels implement privacy-by-design and by-default (Art. 25 GDPR [43], interpreted per the EDPB Guidelines 4/2019 [44]) at the evidence layer: secrets and classified entries are excluded from retrieval surfaces unless explicitly overridden.
- **Right to erasure as file deletion.** Because all state is local files (runs, context database, uploads), erasure is complete and verifiable by deletion, a benefit of the local-first stance.

7.5.2. AI regulation and risk management

The EU Artificial Intelligence Act (Regulation (EU) 2024/1689) [45] classifies AI systems by risk. *thelab-mcp* is a development tool in which LLMs propose and interpret, but decisions about training and serving are deterministic and human-approved; it does not make autonomous decisions about natural persons. Its most plausible classification is as a limited-risk/general-purpose-AI *consumer* (the system calls external LLM providers subject to their own obligations), with transparency obligations satisfied by design. Agent activity is logged and proposals are attributable. Complementing the legal frame, the system's risk posture was mapped against the NIST AI Risk Management Framework and its Generative AI profile [46], [47]: the "govern" function is realized by the approval boundary and audit trail, "measure" by the automated evaluator and agent reliability metrics, and "manage" by the redaction, sandbox, and scale-guard controls.

7.5.3. Licenses and intellectual property

- **Software.** The repository is distributed under the MIT license; all runtime dependencies are permissively licensed (MIT/BSD/Apache-2.0), verified at the lock level. No copyleft obligation is propagated by the stack.
- **Datasets.** Kaggle datasets carry a per-dataset license. The ingestion adapter records the source slug and page context in the context pack, giving each experiment a traceable provenance record. The thesis benchmarks use publicly documented datasets with attribution.
- **Originality.** The system is original work; third-party standards it implements: MCP, JSON-RPC, nbformat, the OpenAI-compatible API convention are open specifications, and no patentable industrial property is claimed. The work is published under Creative Commons BY-NC-ND for the document and MIT for the code, per the cover page.

7.5.4. Technical standards

The work implements or conforms to: the Model Context Protocol specification (JSON-RPC 2.0 over stdio) [11]; the JSON schema dialect used by MCP tool declarations; nbformat (Jupyter notebook schema) for generated notebooks [48]; SQL via SQLite with the FTS5 extension [15]; and HTTP/1.1 with server-sent events for the service layer. Python code follows PEP 8 as enforced by the ruff configuration (line length 100, rule sets `E,F,I,UP,W,B`) and is statically typed under a semi strict mypy configuration.

7.6. Socio-economic environment

7.6.1. Budget

The project was developed on existing personal hardware, a consumer laptop with 16 GB RAM and no GPU. So its capital cost is zero marginal. The development tooling costs are recorded in the project log and summarized in Table 7.1. The production system itself costs *nothing per run* when run fully deterministically and offline. The optional LLM usage is bounded by the adapter configuration, for example a fully local Ollama setup has zero marginal cost.

Item	Cost	Notes
Hardware (existing laptop, AMD Ryzen 7 3700U, 16 GB RAM, no GPU)	EUR 0	Commodity CPU; all experiments ran on it
Opencode (coding assistant agent)	~EUR 60	Main development tool
OpenRouter (model access)	~EUR 40	Agentic testing and auditing across providers
Perplexity (research assistant)	~EUR 20	Research and idea exploration
VS Code / Zed (editors)	EUR 0	Free licenses
Ollama (local models)	EUR 0	llama3.2:3b / qwen3:4b served locally
Total (development)	**~EUR 120**	**Recorded over the project window**

Table 7.1. DEVELOPMENT BUDGET (RECORDED COSTS OVER THE PROJECT WINDOW).

7.6.2. Socio-economic impact analysis

- **Economic.** The system demonstrates that a complete, auditable ML factory runs on commodity hardware with zero per-experiment cloud cost. For students, researchers, and small organizations unable to justify expensive cloud MLOps stacks and several agentic subscriptions. A system like this can lower the entry barrier of ML practice to the cost of the electricity consumed. The recorded budget above doubles as evidence; most of the costs were code-assisted generation tasks and intensive testing, not the system's internal operations.
- **Social.** The approval boundary and the evidence trail give non-expert users a supervised path to agentic ML. The human approves, the factory executes, and every claim is checkable. This is a concrete answer to the trust problem of many opaque AI-generated analytics. The trade-off is also acknowledged; local-first tools shift the security and backup responsibilities entirely to the user, creating a genuine risk if the local workspace is poorly secured. The honest audit trail and the documented sandbox limits are the mitigations this project contributes.
- **Environmental.** Registered model families are intentionally lightweight, scale guards reject infeasible runs in milliseconds, and no GPU compute was used, keeping the training footprint minimal. The genuine trade-off lies in the agent layer. While local execution avoids hyperscale datacenter overhead, the development phase used LLM API calls that still carry real provider side energy and water costs. Arguably, a local, gated, small-batch system ensures resource consumption remains transparent, measurable, and user-controlled, contrasting sharply with opaque cloud inference with uninspectable costs.

- **Ethical.** The system architecture separates proposing from execution, keeping a human meaningfully in the loop for consequential decisions. Complementing this, the redaction layer prevents the evidence store from leaking secrets, while the thesis documents system failures such as agent errors, audit findings, and rejected runs with the same rigor as successes. Not making overclaims of the agentic features. The health, genomics, and financial datasets used in stress experiments serve as analysis proxies to test the data-to-model pipeline on realistic, high-stakes tabular data. No model produced by this system is intended for clinical, diagnostic, or financial decision-making.

- **Exploitation path.** The repository is open-sourced (MIT) as a reference architecture and a usable tool. The possible adoption paths are education (an inspectable agentic-ML system to teach from), privacy-sensitive individual analytics, and as scaffolding for MCP-based agent ecosystems. Each of them evaluated with recorded evidence.

8. CONCLUSIONS AND FUTURE WORK

8.1. Conclusions

This thesis set out to demonstrate that a local, multi-agent orchestration architecture built on typed contracts and the Model Context Protocol can execute a reproducible data-to-model workflow and expose its artifacts to independent agents without coupling to any specific LLM provider. The work supports this claim with a complete implementation and a recorded evaluation. Achieving the objectives proposed at the start of the project:

- **O1 (Deterministic factory).** The pipeline successfully turns a local CSV into a registered, locally servable model with a complete eleven-artifact evidence set per run, with rejection treated as a first-class outcome with full provenance, validation at real scale (164k rows), and protection by traceable scale guards.
- **O2 (Typed contracts), achieved.** Six strict Pydantic contracts form the system's shared language; the strictness rejected integration errors and agent mistakes alike, converting both into readable validation messages.
- **O3 (MCP surface).** Seven stdio servers with 32 schema-bounded tools that expose datasets, profiles, contracts, models, artifacts, context, EDA skills, and orchestration, with RQ2 and Experiment 1 verifying independent consumption end-to-end.
- **O4 (Context store)** The SQLite+FTS5 store provides redacted, fingerprinted, privacy-leveled evidence with provably read-only retrieval (RQ3). The case study shows it functioning as genuine agent memory.
- **O5 (Multi-Agentic layer).** Organized around a single typed protocol abstracting four inference provider adapters. Role-specialized agents with distinct prompt contracts. An orchestrator and grounded chat both operating through a single approval chokepoint with claim-level grounding. Agent-initiated execution initiated to `awaiting_approval`, with rejected proposals being unexecutable. Sandboxed exploration over the deterministic baseline's artifacts, and provider-free runs on the mock adapter, proving provider-agnosticism structurally.
- **O6 (Evaluation)** The three system-deterministic questions (RQ1–RQ3) were passed under an automated evaluator (inside a disposable temporary workspace) at every milestone of the project. The four recorded experiments: three-ways-in with the provider dimension, S&P-scale validation, the grounded-autonomy agentic round and the multi-domain stress validation extended the evidence to realistic data. The agentic research questions (RQ4–RQ6) were evaluated across ten datasets and 37

agentic generations. Resulting in: 19 replay-verified absorptions (17 distinct champion models), 15 of them from rounds with 100% batch validity, 6 honest refusals, and the mixed-model team outperforming every uniform team.

One scope note belongs in the conclusions. The factory is deliberately tabular, CPU-only, and local. Deep learning, GPU training, and distributed execution were taken out of the project scope by recorded decisions. The contribution is not model sophistication but the auditable orchestration protocol around the models. Measured by system properties (reproducibility, interoperability and evidence integrity) rather than by benchmark leadership. Capability benchmarks over the agent layer are the natural continuation of the project's evolution (Section 8.2).

8.2. Future work

The following extensions are concrete future implementations, ordered by the expected impact they produce:

1. **OS-level sandbox confinement.** Close the gap with UID separation or kernel confinement (landlock/nsjail), upgrading the sandbox from compute isolation to filesystem confinement.
2. **Grammar-constrained sampling.** Replace the post-hoc JSON repair with provider-side structured outputs, where the JSON schema is enforced while the model is still generating. This removes the repair ladder at its root cause, making every agent answer schema-valid by construction.
3. **Semantic hybrid retrieval.** Add an optional local embedding index alongside FTS5, preserving the redaction and immutability guarantees, to recover paraphrase recall without cloud vector stores and whose design can mimic graph-structured memory approaches for code and knowledge bases [49], [50].
4. **Unsupervised learning.** Expand the scope to accept unsupervised learning tasks such as cluster-profiling and dimensionality reduction.
5. **HTTP MCP transport.** Serve the same tool surface over HTTP+SSE to reach remote MCP clients, with the existing path-safety and schema layers unchanged.
6. **Live-provider automated evaluation.** Extend the evaluator with a provider-configurable profile so RQ-level checks can be exercised over Ollama/OpenRouter end-to-end.
7. **Capability-benchmark evaluation.** Run an external ML-agent capability benchmark (e.g., MLE-bench's suite [25]) over the agent layer as a complementary evaluation to the system-property checks of this thesis (Section 2.5.1).

8. **Notebook execution in the sandbox.** Execute generated `report.ipynb` cells sequentially in the sandbox with per-cell state, closing the loop from artifact to verified reproduction.

9. **The human–machine learning cycle.** Persist agent sessions as learning material, generated exercises from the user's own conversations, so both sides can improve recursively.

10. **Agent self-improvement loop.** Feed experiment outcomes and rejection reasons back into proposal strategies with the same approval boundary (the case study's self-correction, generalized).

8.3. Closing remark

The interesting question for agentic ML is not how capable the model is, but how much the surrounding system can prove about what happened. The architecture of *thelab-mcp* is built on typed contracts, deterministic execution, redacted memory, protocol-bounded interoperability, a single approval gate, and rejections kept as evidence. It proves that this level of verification can run on a compact system. The system-level research questions are answered automatically on every execution, and every agentic claim is held to a predefined pass bar. This is the thesis's primary practical contribution.

BIBLIOGRAPHY

[1] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[2] T. B. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[3] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.

[4] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[5] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[6] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[7] R. Sutton, *The bitter lesson*, http://www.incompleteideas.net/IncIdeas/BitterLesson.html [Accessed: Sep. 2026], 2019.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[9] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[10] J. S. Park et al., "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[11] Anthropic, *Model context protocol: Specification*, https://modelcontextprotocol.io/specification [Accessed: Sep. 2026], 2024.

[12] Anthropic, *Introducing the model context protocol*, https://www.anthropic.com/news/model-context-protocol [Accessed: Sep. 2026], 2024.

[13] D. Horthy, *12-factor agents: Principles for building reliable LLM applications*, https://github.com/humanlayer/12-factor-agents [Accessed: Sep. 2026], 2025.

[14] B. García, *Context engineering: Companion repository (system prompts, tools/MCP, memory, evaluation)*, https://github.com/bonigarcia/context-engineering [Accessed: Sep. 2026], 2025.

[15] SQLite Consortium, *SQLite FTS5 Extension: Full-text search*, https://www.sqlite.org/fts5.html [Accessed: Sep. 2026], 2024.

[16] J. Pineau et al., "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[17] M. Zaharia et al., *Accelerating the machine learning lifecycle with MLflow*, IEEE Data Engineering Bulletin, 2018.

[18] Iterative, Inc., *DVC: Data version control — git for data science models and datasets*, https://dvc.org/ [Accessed: Sep. 2026], 2024.

[19] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[20] Q. Wu et al., *AutoGen: Enabling next-gen llm applications via multi-agent conversation*, arXiv:2308.08155, 2023.

[21] LangChain, Inc., *LangGraph: Building stateful, multi-actor applications with llms*, https://langchain-ai.github.io/langgraph/ [Accessed: Sep. 2026], 2024.

[22] CrewAI Inc., *CrewAI: Framework for orchestrating role-playing, autonomous AI agents*, https://docs.crewai.com/ [Accessed: Sep. 2026], 2024.

[23] Pydantic Services Inc., *PydanticAI: Agent framework with typed, structured outputs*, https://ai.pydantic.dev/ [Accessed: Sep. 2026], 2024.

[24] Microsoft, *Semantic kernel: Integrate cutting-edge LLM technology quickly into your apps*, https://github.com/microsoft/semantic-kernel [Accessed: Sep. 2026], 2024.

[25] J. S. Chan et al., "MLE-bench: Evaluating machine learning agents on machine learning engineering," in *arXiv preprint arXiv:2410.07095*, 2024.

[26] Y. Lai et al., "DS-1000: A natural and reliable benchmark for data science code generation," in *International Conference on Machine Learning (ICML)*, 2023.

[27] L. Jing et al., "DSBench: How far are data science agents from becoming data science experts?" In *arXiv preprint arXiv:2409.07703*, 2024.

[28] TML-Bench authors, "TML-Bench: Benchmark for data science agents on tabular ML tasks," *arXiv preprint arXiv:2603.05764*, 2026.

[29] D. Matthews et al., *AstaBench: Rigorous benchmarking of AI agents for science*, Allen Institute for AI, https://allenai.org/asta/bench [Accessed: Sep. 2026], 2025.

[30] S. Colvin et al., *Pydantic: Data validation using Python type hints*, https://docs.pydantic.dev/ [Accessed: Sep. 2026], 2024.

[31] Ollama, *Ollama: Get up and running with large language models locally*, https://ollama.com/ [Accessed: Sep. 2026], 2024.

[32] Open Source Security Foundation, *Concise guide for developing more secure software*, https://openssf.org/educational-resources/ [Accessed: Sep. 2026], 2024.

[33] OWASP Foundation, *MCP security cheat sheet*, https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html [Accessed: Sep. 2026], 2025.

[34] National Security Agency, *Model context protocol (MCP): Security design considerations for AI ecosystems*, Cybersecurity Information Sheet, 2025.

[35] OWASP Foundation, *OWASP top 10 for large language model applications*, https://owasp.org/www-project-top-10-for-large-language-model-applications/ [Accessed: Sep. 2026], 2024.

[36] Breaking the Protocol authors, "Breaking the protocol: Security analysis of the model context protocol specification and prompt injection vulnerabilities in tool-integrated LLM agents," *arXiv preprint arXiv:2601.17549*, 2026.

[37] Hou, X. and others, "Model context protocol (MCP): Landscape, security threats, and future research directions," *ACM Computing Surveys*, 2026, doi:10.1145/3796519.

[38] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[39] S. Ramírez, *Fastapi framework*, https://fastapi.tiangolo.com/ [Accessed: Sep. 2026], 2024.

[40] Kaggle, *Kagglehub: The official Kaggle python library for dataset access*, https://github.com/Kaggle/kagglehub [Accessed: Sep. 2026], 2024.

[41] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference (SciPy)*, 2010, pp. 56–61.

[42] Anthropic and contributors, *Model context protocol: Transports (streamable http)*, https://modelcontextprotocol.io/specification/draft/basic/transports [Accessed: Sep. 2026], 2025.

[43] European Parliament and Council of the European Union, *Regulation (eu) 2016/679 of 27 april 2016 (general data protection regulation)*, Official Journal of the European Union, L 119, 2016.

[44] European Data Protection Board, *Guidelines 4/2019 on article 25 data protection by design and by default*, Version 2.0, adopted 20 October 2020, 2020.

[45] European Parliament and Council of the European Union, *Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act)*, Official Journal of the European Union, 2024.

[46] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," U.S. Department of Commerce, Tech. Rep. NIST AI 100-1, 2023.

[47] National Institute of Standards and Technology, "Generative artificial intelligence profile (NIST AI 600-1)," U.S. Department of Commerce, Tech. Rep., 2024.

[48] Jupyter Development Team, *Nbformat: Reference implementation of the Jupyter notebook format*, https://nbformat.readthedocs.io/ [Accessed: Sep. 2026], 2024.

[49] mempalace contributors, *Memory palace: Structured memory access for LLM agents*, https://github.com/milla-jovovich/mempalace [Accessed: Sep. 2026], 2026.

[50] DeusData, *Codebase-memory-MCP: From codebase to graph*, https://github.com/DeusData/codebase-memory-mcp [Accessed: Sep. 2026], 2026.

A. REPRODUCTION GUIDE

This annex is a step-by-step guide to reproduce every result in this thesis from a fresh clone of the repository. All commands assume a Unix-like shell from the repository root.

A.1. Environment setup

CODE A.1. Install from the lockfile.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.lock
pip install -e .
```

Supported Python is ≥3.11; the lockfile pins every runtime dependency.

A.2. Reproducible training (RQ1 surface)

CODE A.2. Direct deterministic run.

```
thelab run model --dataset examples/iris.csv --target species \
  --model logistic_regression --seed 42 --output runs
# Run completed: run-<timestamp>-<id>
#   Test accuracy: 1.000000   Test macro F1: 1.000000
```

Re-running the command yields a new `run_id` with equivalent metrics; `runs/<run_id>/manifest.json` carries seed, config, and dependency provenance (Annex B).

A.3. The three ways in (Experiment 1)

CODE A.3. HTTP and MCP paths to the identical run.

```
# HTTP
thelab-model-service --port 8000
curl -s -X POST localhost:8000/train -H 'Content-Type: application/
    json' \
  -d '{"dataset_id": "fixtures/iris.csv", "target": "species",
       "model": "logistic_regression", "seed": 42}'

# MCP (independent client)
thelab-mcp-demo model_registry --run-id <run_id>
```

A.4. Context store (RQ3 surface)

CODE A.4. Index and search local evidence.

```
thelab context index --source .thelab/local-logs/agent-events.jsonl
thelab context search "reproducibility" --limit 10
thelab context show <event_id>
```

A.5. Automated evaluation

CODE A.5. Run the thesis evaluator (disposable workspace; exit 0 on PASS).

```
PATH=.venv/bin:$PATH python scripts/evaluate_thesis.py
```

A.6. Quality gates

CODE A.6. Full quality suite.

```
.venv/bin/ruff check thelab tests scripts
.venv/bin/mypy thelab
.venv/bin/python -m pytest tests/ -q
```

A.7. Real-data and Kaggle paths

CODE A.7. Real-data path: upload, EDA, clean, orchestrate (HTTP surface).

```
# Upload the CSV in the UI (Data view), run EDA, click Clean dataset,
# then start an experiment in the Experiments view and watch the
# live stage pipeline; inspect best-run metrics in History.
thelab ingest kaggle <owner>/<dataset-slug>   # Kaggle ingestion via
    CLI
```

A.8. Troubleshooting

The documented CLI troubleshooting table, reproduced from the user guide:

Symptom	Cause	Fix
`dataset must be a relative path`	Absolute path passed	Run from the workspace root
`unsupported model`	Name not in registry	See Table 3.2
`model 'X' is limited to N training rows`	Scale guard	Scalable model or subsample
`cannot stratify`	Too few samples per class	More rows or fewer classes
`feature columns contain infinite values`	`Inf` in features	Clean the dataset first
`constant feature columns found`	Single-valued feature	Fix data upstream (or use cleaning)
Service returns 404	Run not completed/approved	Check `manifest.json` statuses
Job stuck `running`	Long stage	Watch pipeline; use Cancel

Table A.1. CLI TROUBLESHOOTING (FROM `DOCS/CLI_GUIDE.MD`).

B. RUN MANIFEST EXAMPLE

A real manifest from `runs/` (abridged to the first artifact reference; hashes and identifiers are unmodified).

CODE B.1. Real `manifest.json` (abridged).

```
{
  "run_id": "run-20260829-145048-ee5f986e",
  "input_hash": "925e672a920734ec9f203cf1b7fe18f218520e4cd...",
  "training_config": {
    "model": "logistic_regression",
    "seed": 44,
    "task_type": "classification",
    "preprocessing": ["StandardScaler"],
    "split": {"strategy": "train_test_split", "test_size": 0.2,
              "random_state": 44, "stratify": true,
              "train_count": 91015, "test_count": 22754},
    "dependency_versions": {
      "python": "3.14.6", "thelab": "0.1.0",
      "pydantic": "2.13.4", "pandas": "3.0.5",
      "scikit-learn": "1.9.0", "numpy": "2.5.2",
      "joblib": "1.5.3"
    },
    "estimator": {"class": "LogisticRegression",
                  "module": "sklearn.linear_model._logistic",
                  "default_params": {"max_iter": 200,
                                     "solver": "lbfgs"},
                  "hyperparameters": null},
    "supports_probability": true
  },
  "random_seed": 44,
  "started_at": "2026-08-29T14:50:48.405884Z",
  "finished_at": "2026-08-29T14:50:49.868901Z",
  "final_status": "completed",
  "validation_status": "approved",
  "artifact_refs": [
    {"artifact_id": "run-20260829-145048-ee5f986e-task_spec",
     "artifact_type": "task_spec",
     "relative_path": "task_spec.json",
     "content_hash": "6690192e7a5ea40aa79ed907e982aac3d...",
     "origin": "orchestrator",
     "parent_run_id": "run-20260829-145048-ee5f986e"}
    /* ... ten further artifact references ... */
  ]
}
```

The corresponding `metrics.json`: train accuracy 0.6081 / test accuracy 0.6000, train

macro-F1 0.6032 / test macro-F1 0.5946 (113,769-row dataset): a modest, honest result whose value lies in its provenance.

C. GENERATED NOTEBOOK EXCERPT

Every completed run can generate a reproducible research notebook artifact (`report.ipynb`) over `GET /runs/{id}/notebook`. The generated structure, with the exact cell purposes:

CODE C.1. Generated notebook structure (six cells, nbformat 4).

```
Cell 1 (markdown)  Title + manifest summary
                   run id, model, seed, dataset, status
Cell 2 (code)      Dataset load
                   loads the CSV recorded in inputs.json
Cell 3 (code)      Reproduce
                   run_model(...) with the exact recorded parameters
Cell 4 (code)      Metrics + validation report display
Cell 5 (code)      Artifacts index (model_card.md, contracts)
Cell 6 (markdown)  Findings
                   sub-agent interpretations / context notes
```

The notebook is standards-compliant (nbformat 4) so it opens in Jupyter unchanged; executing cells 2–3 reproduces the run's computation from the recorded parameters. All three P3.7 proof runs (Table 6.5) produced valid notebooks through the endpoint, verified live.

D. EXPERIMENT 4 DATASET TABLE

The Experiment 4 dataset set: Arm A (agentic re-run over datasets with recorded deterministic baselines already stored in `uploads/`) and Arm B (new stress domains where a trained model is genuinely useful), plus the clustering/graph track. All slugs verified live on Kaggle at submission time; targets confirmed at run time; sizes are the stored analysis CSVs per the storage policy of Section 5.7.

Kaggle slug	**Domain**	**Task**	**Arm / tier**
`shrutimechlearn/churn-modelling`	banking	classification	A (recorded)
`camnugent/california-housing-prices`	housing	regression	A (recorded)
`pavansubhasht/ibm-hr-analytics-attrition-dataset`	HR	classification	A (recorded)
`erfan4524/e-commerce-sales-data-analysis-and-eda`	e-commerce	regression	A (recorded)
`yasserh/titanic-dataset`	classic	classification	A (recorded)
`sp_500 analyst ratings (local)`	finance	classification	A (recorded)
`kevinarvai/clinvar-conflicting`	genomics (DNA variants)	classification	B / must
`rabieelkharoua/chronic-kidney-disease-dataset-analysis`	health	classification	B / must
`sudalairajkumar/cryptocurrencypricehistory`	crypto	direction classif.	B / must
`robikscube/hourly-energy-consumption`	electrical usage	regression	B / must
`shivamb/real-or-fake-fake-jobposting-prediction`	NLP/AI	featurize + classif.	B / strong
`andradaolteanu/gtzan-dataset-music-genre-classification`	music signals (feature CSV)	classification	B / strong
`mczielinski/bitcoin-historical-data`	crypto (1-min bars)	scale-stress, regression	B / stretch
`rahman4li/patch-bay-the-mcp-server-ecosystem-graph`	MCP ecosystem	graph + clustering	B / stretch
`adarsh1077/huggingface-hub-50k-ai-models`	AI models	clustering	B / stretch

Table D.1. EXPERIMENT 4 DATASETS (KAGGLE SLUGS, VERIFIED LIVE; TIERED).

Notes: the Patch Bay dataset ships as two CSVs (`nodes.csv` with GitHub statis-

tics and network-centrality features per MCP server; `edges.csv` as a weighted shared-category edge list) and enters the workspace as two ingests from one slug. GTZAN's heavy raw audio stays in the transient kagglehub cache; only the extracted-features CSV is stored. The S&P 500 analyst dataset was downloaded from a public source outside Kaggle and is already recorded in `uploads/`.

D.1. Declaración de uso de Inteligencia Artificial Generativa

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG) EN EL TRABAJO DE FIN DE GRADO (TFG)

He usado IAG en mi TFG

Marca lo que corresponda:

SI	**NO**

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: la evaluación de tu TFG puede verse comprometida si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad).

<table>
<tr><th colspan="2">Pregunta</th></tr>
<tr><td colspan="2">1. En mi interacción con herramientas de IAG he facilitado datos de carácter confidencial contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).</td></tr>
<tr><td>SÍ, he usado estos datos con la autorización de los interesados</td><td>NO, no he usado datos de carácter confidencial</td></tr>
<tr><td colspan="2">2. En mi interacción con herramientas de IAG he facilitado materiales protegidos por derechos de autoría contando siempre con la autorización de los respectivos titulares.</td></tr>
<tr><td>SÍ, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley:

● obra en dominio público
● obra licenciada (licencias Creative Commons)</td><td>NO, no he usado materiales protegidos por derechos de autoría</td></tr>
</table>

• uso de fragmentos con fines de investigación (derecho de cita)	
3. En mi interacción con herramientas de IAG he facilitado **datos de carácter personal** con la debida autorización de los interesados.	
SÍ, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad	NO, no he usado datos de carácter personal
4. Mi utilización de la herramienta de IAG ha **respetado sus términos de uso**, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.	
SI	NO

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IAG. Incluye un ejemplo por cada tipo de uso realizado donde se indique: *[Añade un ejemplo].*

Declaro haber hecho uso del sistema de IAG (Nombre del sistema/herramienta IAG y versión: ChatGPT, Gemini, Copilot…) **para:**

Documentación y redacción:

- *Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IAG*

[Añade un ejemplo] Por ejemplo: He solicitado una lista de alternativas para abordar el problema planteado al inicio del trabajo.

- *Revisión o reescritura de párrafos redactados previamente*

[Añade un ejemplo] Por ejemplo: He solicitado la reescritura de párrafos para acortar las conclusiones manteniendo las mismas ideas ya redactadas.

- *Búsqueda de información o respuesta a preguntas concretas*

[Añade un ejemplo] Por ejemplo: Las preguntas realizadas se orientaban a [encontrar definiciones, explicaciones de procesos, ejemplos de uso, listas de riesgos de uso…]

- *Búsqueda de bibliografía*
- *Resumen de bibliografía consultada*
- *Traducción de texto consultados*

Desarrollar contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

[Añade un ejemplo del flujo habitual de uso para programar] Por ejemplo: He pedido explicaciones sobre un código en lenguaje Python.

- *Generación de esquemas, imágenes, audios o vídeos*

[indicar su uso en este apartado, así como mediante una nota al pie de la figura/elemento generado por IAG] Por ejemplo: Fuente: elaboración propia con ayuda de Midjourney.

- *Procesos de optimización*

[Añade un ejemplo del flujo habitual de uso para optimizar] Por ejemplo: he solicitado ayuda para corregir y explicar errores de un código que no funcionaba correctamente.

- *Tratamiento de datos: recogida, análisis, cruce de datos…*

[Añade un ejemplo] Por ejemplo: una vez recogidos los datos, he pedido una guía en la normalización de los datos para la preparación del posterior análisis.

- *Inspiración de ideas en el proceso creativo*

[Añade un ejemplo] Por ejemplo: una vez he tenido claro el objetivo del trabajo he solicitado ideas para abarcar el proyecto desde un enfoque multidisciplinar.

- *Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo*

Si crees necesario incluirlo, añade, en cada uno de los casos:

empleando el *prompt* …
(escribe la petición realizada a la IAG)
teniendo como interacción …
(describe qué interacción realizaste con la IAG tras la respuesta del prompt)

Ejemplo: ***Declaro*** *haber hecho uso del sistema de IAG* ***Chat GPT X.x*** *para* ***buscar información*** *empleando* ***el prompt: "Dime un ejemplo concreto que ilustre la aplicación de la propuesta urbanística de la Ciudad de los 15 minutos en España"*** *teniendo como interacción* ***la proposición del ejemplo concreto del distrito de Chamartín que se ha reflejado en la memoria****.*

Parte 3: reflexión sobre utilidad

Aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IAG en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

D.2. Declaración de uso de Inteligencia Artificial Generativa (IAG) en el Trabajo de Fin de Grado (TFG)

He usado IAG en mi TFG.

Marca lo que corresponda: **SI.**

Parte 1: declaración sobre comportamiento legal, ético y responsable

En mi interacción con herramientas de IAG he facilitado datos de carácter confidencial contando siempre con la debida autorización de los interesados.

NO, no he usado datos de carácter confidencial.

En mi interacción con herramientas de IAG he facilitado materiales protegidos por derechos de autoría contando siempre con la autorización de los respectivos titulares.

NO, no he usado materiales protegidos por derechos de autoría.

En mi interacción con herramientas de IAG he facilitado datos de carácter personal con la debida autorización de los interesados.

NO, no he usado datos de carácter personal.

Mi utilización de la herramienta de IAG ha respetado sus términos de uso, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado.

SI.

Parte 2: declaración de uso técnico

Declaro haber hecho uso del sistema de IAG (GLM-5.3-flash, Llama-3.3-70b-instruct, Mistral-Small-24b, DeepSeek-V4-flash, Claude-Haiku-4.5, GPT-5.6-luna y Qwen3-4b, vía OpenRouter y Ollama local) como objeto de estudio del propio trabajo, y para: investigar recursos en línea, asistir en tareas de código, realizar auditorías de código y tests.

Documentación y redacción:

Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IAG

Prompt: [Analiza por qué el equipo mixto (analista ollama, ingeniero de características GLM, selector mistral) obtuvo mayor consistencia que los equipos uniformes, y qué alternativas de diseño existen], teniendo como interacción el análisis de alternativas de arquitectura (equipos mixtos frente a uniformes).

Revisión o reescritura de párrafos redactados previamente

Prompt: [Verifica que estas cifras y esta redacción coinciden con los registros generados por el sistema y presenta los resultados vía chat], teniendo como interacción la corrección de cifras y redacción de los capítulos 5 y 6 sobre borradores previamente escritos por mí, con revisión y aceptación manual de cada cambio.

Búsqueda de información o respuesta a preguntas concretas:

Prompt: [Busca repositorios open-source parecidos a MLflow], para encontrar repositorios de código libre como fuente de inspiración para el desarrollo del sistema.

Búsqueda de bibliografía

NO, no se utilizó directamente para una lista curada de fuentes, sino como herramienta de investigación para encontrar recursos útiles algunos de ellos representados en la bibliografía.

Resumen de bibliografía consultada

NO, no se utilizó IAG para resumir la bibliografía.

Traducción de textos consultados

NO, no se utilizó IAG para traducción. La memoria está redactada en inglés por mí y para momentos puntuales se usaron traductores como DeepL.

Desarrollar contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

Asistencia en el desarrollo de líneas de código (programación):

Declaro haber hecho uso de los sistemas de IAG mediante OpenRouter y agentes de programación vía OpenCode para el desarrollo de ciertas partes del código y para la depuración de la aplicación.

Prompt: [Ayúdame a entender por qué da el error de la línea 124 al compilar el código]

Generación de esquemas, imágenes, audios o vídeos:

Prompt: [Genera una tabla usando matplotlib a partir de los esquemas JSON generados por la aplicación]. Para la creación de tablas y gráficas a partir de contexto generado por la propia aplicación.

Procesos de optimización:

Prompt: [Actúa como un auditor de código profesional, busca vulnerabilidades y fallos estructurales del código]. Para procesos de auditoría de código y sugerencias de refactorización.

Tratamiento de datos: recogida, análisis, cruce de datos:

Prompt: [Analiza estos datos de manera exhaustiva y prepara un plan de EDA; posteriormente te presentaré mi plan establecido para realizar una comparación]. Para investigar datasets antes de usarlos como input para la aplicación desarrollada.

Inspiración de ideas en el proceso creativo:

Prompt: [Analiza mis notas desestructuradas, ayúdame a realizar un plan coherente dividido en pasos]. Para la validación de ideas antes de convertirlas en código.

Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo:

Para el propio trabajo en sí. Una arquitectura multiagente orquestadora de flujos de aprendizaje automático.

Parte 3: reflexión sobre utilidad

La IA ha formado una parte indiscutible del trabajo, siendo este el principal tema de investigación: la creación de una arquitectura usando diferentes agentes enfocada al análisis de datos y la generación de modelos de aprendizaje automático (ML). Además de ayudar como asistente externo en las tareas de investigación de recursos, validación de ideas y desarrollo de código en partes del repositorio de la aplicación.